

typora-copy-images-to: 尚硅谷 宋红康 深入解读Java12&13新特性_配图

深入解读 Java 12 & 13 新特性

讲师：宋红康

微博：尚硅谷宋红康



一、关于Java生态圈

Java是目前应用最为广泛的软件开发平台之一。随着Java以及Java社区的不断壮大，Java 也早已不再是简简单单的一门计算机语言了，它更是一个平台、一种文化、一个社区。

作为一个平台，Java虚拟机扮演着举足轻重的作用。除了Java语言，任何一种能够被编译成字节码的计算机语言都属于Java这个平台。Groovy、Scala、JRuby、Kotlin等都是Java平台的一部分，它们依赖于Java虚拟机，同时，Java平台也因为它们变得更加丰富多彩。

作为一种文化，Java几乎成为了“开源”的代名词。在Java程序中，有着数不清的开源软件和框架。如Tomcat、Struts, Hibernate, Spring等。就连JDK和JVM自身也有不少开源的实现，如OpenJDK、Apache Harmony。可以说，“共享”的精神在Java世界里体现得淋漓尽致。

作为一个社区，Java拥有全世界最多的技术拥护者和开源社区支持，有数不清的论坛和资料。从桌面应用软件、嵌入式开发到企业级应用、后台服务器、中间件，都可以看到Java的身影。其应用形式之复杂、参与人数之众多也令人咋舌。可以说，Java社区已经俨然成为了一个良好而庞大的生态系统。**其实这才是Java最大的优势和财富。**

二、Java 老矣，尚能饭否？

看看最新的TIOBE 社区的语言热度排行榜：

Sep 2019	Sep 2018	Change	Programming Language	Ratings	Change
1	1		Java	16.661%	-0.78%
2	2		C	15.205%	-0.24%
3	3		Python	9.874%	+2.22%
4	4		C++	5.635%	-1.76%
5	6	▲	C#	3.399%	+0.10%
6	5	▼	Visual Basic .NET	3.291%	-2.02%
7	8	▲	JavaScript	2.128%	-0.00%
8	9	▲	SQL	1.944%	-0.12%
9	7	▼	PHP	1.863%	-0.91%
10	10		Objective-C	1.840%	+0.33%
11	34	▲▲	Groovy	1.502%	+1.20%
12	14	▲	Assembly language	1.378%	+0.15%
13	11	▼	Delphi/Object Pascal	1.335%	+0.04%
14	16	▲	Go	1.220%	+0.14%
15	12	▼	Ruby	1.211%	-0.08%
16	15	▼	Swift	1.100%	-0.12%
17	20	▲	Visual Basic	1.084%	+0.40%
18	13	▼	MATLAB	1.062%	-0.21%
19	18	▼	R	1.049%	+0.03%

2.1 Java 是最好的语言吗？

不是，因为在每个领域都有更合适的编程语言。

- C 语言无疑是现代计算机软件编程语言的王者，几乎所有的操作系统都是 C 语言写成的。C++ 是面向对象的 C 语言，一直在不断的改进。
- JavaScript 是能运行在浏览器中的语言，丰富的前端界面离不开 Javascript 的功劳。近年来的 Node.js 又在后端占有一席之地。
- Python 用于系统管理，并通过高性能预编译的库，提供 API 来进行科学计算，文本处理等，是 Linux 必选的解释性语言。
- Ruby 强于 DSL（领域特定语言），程序员可以定义丰富的语义来充分表达自己的思想。
- Erlang 就是为分布式计算设计的，能保证在大规模并发访问的情况下，保持强壮和稳定性。
- Go 语言内置了并发能力，可以编译成本地代码。当前新的网络相关项目，很大比例是由 Go 语言编写的，如 Docker、Kubernetes 等。
- 编写网页用 PHP，函数式编程有 Lisp，编写 iOS 程序有 Swift/Objective-C。

一句话概括，能留在排行榜之上的语言，都是好的语言，在其所在的领域能做到最好。

2.2 Java 语言到底有什么优势？

其一，语法比较简单，学过计算机编程的开发者都能快速上手，**Java是一门极好的初学者入门语言**。和C/C++相比，Java在设计上有着绝对的优势，开发人员可以尽快从语言本身的复杂性中解脱出来，将更多的精力投向软件自身的业务功能。

其二，在若干领域都有很强的竞争力，比如服务端编程，高性能网络程序，企业软件事务处理，分布式计算，Android 移动终端、嵌入式设备应用开发等等。

最重要的一点是符合工程学的需求，我们知道现代软件都是协同开发，那么代码可维护性，编译时检查，较为高效的运行效率，跨平台能力，丰富的 IDE，测试，项目管理工具配合。都使得 Java 成为企业软件公司的首选，也得到很多互联网公司的青睐。

其三，没有短板，容易从市场上找到 Java 软件工程师。软件公司选择 Java 作为主要开发语言，再在特定的领域使用其他语言协作编程，这样的组合选择，肯定是有大的问题。

所以综合而言，Java 语言全能方面是最好的。

随着 Java 每半年更新一次的脚步，Java 的新版本中也出现了越来越多与其他语言相似的特性，博采众长的 Java，还能继续保持生机。

但是，Java在不少地方依然受到了广大开发人员的诟病，它烦琐的语法经常受到Python等开发人员的耻笑。在语言的动态性上，甚至也远远不如和它年龄相仿的PHP语言。但为了支持动态语言，Java虚拟机推出了新的函数调用指令 `invokedynamic`，试图弥补Java在动态调用上的不足。

三、JDK 各版本主要特性

JDK Version 1.0

1996-01-23 Oak(橡树)

- 1 初代版本，伟大的一个里程碑，但是是纯解释运行，使用外挂JIT，性能比较差，运行速度慢。

JDK Version 1.1

1997-02-19

- 1 JDBC(Java DataBase Connectivity);
- 2
- 3 支持内部类;
- 4
- 5 RMI(Remote Method Invocation) ;
- 6
- 7 反射;
- 8
- 9 Java Bean;

JDK Version 1.2

1998-12-08 Playground(操场)

```
1  集合框架；
2
3  JIT(Just In Time)编译器；
4
5  对打包的Java文件进行数字签名；
6
7  JFC(Java Foundation Classes)，包括Swing 1.0，拖放和Java2D类库；
8
9  Java插件；
10
11 JDBC中引入可滚动结果集，BLOB，CLOB，批量更新和用户自定义类型；
12
13 Applet中添加声音支持。
```

同时，Sun发布了JSP/Servlet、EJB规范，以及将Java分成了J2EE、J2SE和J2ME。这表明了Java开始向企业、桌面应用和移动设备应用3大领域挺进。

JDK Version 1.3

2000-05-08 Kestrel(红隼)

```
1  Java Sound API；
2
3  jar文件索引；
4
5  对Java的各个方面都做了大量优化和增强；
```

此时，Hotspot虚拟机成为Java的默认虚拟机。

JDK Version 1.4

2002-02-13 Merlin(隼)

```
1  断言；
2
3  XML处理；
4
5  Java打印服务；
6
7  Logging API；
8
9  Java Web Start；
10
11 JDBC 3.0 API；
12
13 Preferences API；
14
15 链式异常处理；
16
17 支持IPV6；
18
```

```
19 支持正则表达式；
20
21 引入Image I/O API
```

同时，古老的Classic虚拟机退出历史舞台。

一年后，Java平台的Scala正式发布，同年Groovy也加入了Java阵营。

JAVA 5

2004-09-30 Tiger(老虎)

```
1  类型安全的枚举；
2
3  泛型；
4
5  自动装箱与自动拆箱；
6
7  元数据(注解)；
8
9  增强循环,可以使用迭代方式；
10
11 可变参数；
12
13 静态引入；
14
15 Instrumentation；
```

同时JDK 1.5改名为J2SE 5.0。

JAVA 6

2006-12-11 Mustang(野马)

```
1  支持脚本语言；
2
3  JDBC 4.0API；
4
5  Java Compiler API；
6
7  可插拔注解；
8
9  增加对Native PKI(Public Key Infrastructure), Java GSS(Generic Security
10 Service),Kerberos和LDAP(Lightweight Directory Access Protocol)支持；
11 继承web Services；
```

- 同年，Java开源并建立了OpenJDK。顺理成章，Hotspot虚拟机也成为了OpenJDK中的默认虚拟机。
- 2007年，Java平台迎来了新伙伴Clojure。
- 2008年，Oracle收购了BEA，得到了JRockit虚拟机。
- 2009年，Twitter宣布把后台大部分程序从Ruby迁移到Scala，这是Java平台的又一次大规模应用。

- 2010年，Oracle收购了Sun，获得最具价值的Hotspot虚拟机。此时，Oracle拥有市场占有率最高的两款虚拟机Hotspot和JRockit，并计划在未来对它们进行整合。

JAVA 7

2011-07-28 Dolphin(海豚)

```
1  钻石型语法(在创建泛型对象时应用类型推断);
2
3  支持try-with-resources(在一个语句块中捕获多种异常);
4
5  switch语句块中允许以字符串作为分支条件;
6
7  引入Java NIO.2开发包;
8
9  在创建泛型对象时应用类型推断;
10
11 支持动态语言;
12
13 数值类型可以用二进制字符串表示,并且可以在字符串表示中添加下划线;
14
15 null值的自动处理;
```

在JDK 1.7中，正式启用了新的垃圾回收器G1，支持了64位系统的压缩指针。

JAVA 8

2014-03-18

```
1  Lambda 表达式 - Lambda允许把函数作为一个方法的参数(函数作为参数传递进方法中)。
2
3  方法引用 - 方法引用提供了非常有用的语法,可以直接引用已有Java类或对象(实例)的方法或构造器。与lambda
   联合使用,方法引用可以使语言的构造更紧凑简洁,减少冗余代码。
4
5  默认方法 - 默认方法就是一个在接口里面有了一个实现的方法。
6
7  新工具 - 新的编译工具,如:Nashorn引擎 jjs、类依赖分析器jdeps。
8
9  Stream API -新添加的Stream API(java.util.stream) 把真正的函数式编程风格引入到Java中。
10
11 Date Time API - 加强对日期与时间的处理。
12
13 Optional 类 - Optional 类已经成为 Java 8 类库的一部分,用来解决空指针异常。
14
15 Nashorn, JavaScript 引擎 - Java 8提供了一个新的Nashorn javascript引擎,它允许我们在JVM上运行特
   定的javascript应用。
```

尚硅谷 Java 8新特性视频观看地址：http://www.atguigu.com/download_detail.shtml?v=6

JAVA 9

2017-09-22

1 模块系统：模块是一个包的容器，Java 9 最大的变化之一是引入了模块系统（Jigsaw 项目）。
2
3 REPL（JShell）：交互式编程环境。
4
5 HTTP 2 客户端：HTTP/2标准是HTTP协议的最新版本，新的 HttpClient API 支持 WebSocket 和 HTTP2 流
以及服务器推送特性。
6
7 改进的 Javadoc：Javadoc 现在支持在 API 文档中的进行搜索。另外，Javadoc 的输出现在符合兼容 HTML5
标准。
8
9 多版本兼容 JAR 包：多版本兼容 JAR 功能能让你创建仅在特定版本的 Java 环境中运行库程序时选择使用的
class 版本。
10
11 集合工厂方法：List, Set 和 Map 接口中，新的静态工厂方法可以创建这些集合的不可变实例。
12
13 私有接口方法：在接口中使用private私有方法。我们可以使用 private 访问修饰符在接口中编写私有方法。
14
15 进程 API：改进的 API 来控制和管理操作系统进程。引进 java.lang.ProcessHandle 及其嵌套接口 Info
来让开发者逃离时常因为要获取一个本地进程的 PID 而不得不使用本地代码的窘境。
16
17 改进的 Stream API：改进的 Stream API 添加了一些便利的方法，使流处理更容易，并使用收集器编写复杂的查
询。
18
19 改进 try-with-resources：如果你已经有一个资源是 final 或等效于 final 变量,您可以在 try-with-
resources 语句中使用该变量，而无需在 try-with-resources 语句中声明一个新变量。
20
21 改进的弃用注解 @Deprecated：注解 @Deprecated 可以标记 Java API 状态，可以表示被标记的 API 将会被
移除，或者已经破坏。
22
23 改进钻石操作符(Diamond Operator)：匿名类可以使用钻石操作符(Diamond Operator)。
24
25 改进 Optional 类：java.util.Optional 添加了很多新的有用方法，Optional 可以直接转为 stream。
26
27 多分辨率图像 API：定义多分辨率图像API，开发者可以很容易的操作和展示不同分辨率的图像了。
28
29 改进的 CompletableFuture API：CompletableFuture 类的异步机制可以在 ProcessHandle.onExit
方法退出时执行操作。
30
31 轻量级的 JSON API：内置了一个轻量级的JSON API
32
33 响应式流（Reactive Streams）API：Java 9中引入了新的响应式流 API 来支持 Java 9 中的响应式编程。

尚硅谷 Java 9新特性视频观看地址：http://www.atguigu.com/download_detail.shtml?v=9

JAVA 10

2018-03-21

1 JEP286，var 局部变量类型推断。
2
3 JEP296，将原来用 Mercurial 管理的众多 JDK 仓库代码，合并到一个仓库中，简化开发和管理过程。
4

5 JEP304, 统一的垃圾回收接口。
6
7 JEP307, G1 垃圾回收器的并行完整垃圾回收, 实现并行性来改善最坏情况下的延迟。
8
9 JEP310, 应用程序类数据 (AppCDS) 共享, 通过跨进程共享通用类元数据来减少内存占用空间, 和减少启动时间。
10
11 JEP312, ThreadLocal 握手交互。在不进入到全局 JVM 安全点 (Safepoint) 的情况下, 对线程执行回调。优化可以只停止单个线程, 而不是停全部线程或一个都不停。
12
13 JEP313, 移除 JDK 中附带的 javah 工具。可以使用 javac -h 代替。
14
15 JEP314, 使用附加的 Unicode 语言标记扩展。
16
17 JEP317, 能将堆内存占用分配给用户指定的备用内存设备。
18
19 JEP317, 使用 Graal 基于 Java 的编译器, 可以预先把 Java 代码编译成本地代码来提升效能。
20
21 JEP318, 在 OpenJDK 中提供一组默认的根证书颁发机构证书。开源目前 Oracle 提供的 Java SE 的根证书, 这样 OpenJDK 对开发人员使用起来更方便。
22
23 JEP322, 基于时间定义的发布版本, 即上述提到的发布周期。版本号为
 \ \$FEATURE.\ \$INTERIM.\ \$UPDATE.\ \$PATCH, 分别是大版本, 中间版本, 升级包和补丁版本。

JAVA 11

2018-09-25

1 181: Nest-Based Access Control (基于嵌套的访问控制)
2
3 309: Dynamic Class-File Constants (动态的类文件常量)
4
5 315: Improve Aarch64 Intrinsics (改进 Aarch64 Intrinsics)
6
7 318: Epsilon: A No-Op Garbage Collector (Epsilon 垃圾回收器, 又被称为 "No-Op (无操作)" 回收器)
8
9 320: Remove the Java EE and CORBA Modules (移除 Java EE 和 CORBA 模块, JavaFX 也已被移除)
10
11 321: HTTP Client (Standard)
12
13 323: Local-Variable Syntax for Lambda Parameters (用于 Lambda 参数的局部变量语法)
14
15 324: Key Agreement with Curve25519 and Curve448 (采用 Curve25519 和 Curve448 算法实现的密钥协议)
16
17 327: Unicode 10
18
19 328: Flight Recorder (飞行记录仪)
20
21 329: ChaCha20 and Poly1305 Cryptographic Algorithms (实现 ChaCha20 和 Poly1305 加密算法)
22
23 330: Launch Single-File Source-Code Programs (启动单个 Java 源代码文件的程序)
24

- 25 331: Low-Overhead Heap Profiling (低开销的堆分配采样方法)
- 26
- 27 332: Transport Layer Security (TLS) 1.3 (对 TLS 1.3 的支持)
- 28
- 29 333: ZGC: A Scalable Low-Latency Garbage Collector (Experimental) (ZGC: 可伸缩的低延迟垃圾回收器, 处于实验性阶段)
- 30
- 31 335: Deprecate the Nashorn JavaScript Engine (弃用 Nashorn JavaScript 引擎)
- 32
- 33 336: Deprecate the Pack200 Tools and API (弃用 Pack200 工具及其 API)

尚硅谷 Java 11 新特性视频观看地址: http://www.atguigu.com/download_detail.shtml?v=128

四、Java 发布计划

4.1 JDK 各版本支持周期

Oracle Java SE Support Roadmap ^{*†}				
Release	GA Date	Premier Support Until	Extended Support Until	Sustaining Support
6	December 2006	December 2015	December 2018	Indefinite
7	July 2011	July 2019	July 2022 ^{*****}	Indefinite
8 ^{**}	March 2014	March 2022	March 2025	Indefinite
9 (non-LTS)	September 2017	March 2018	Not Available	Indefinite
10 (non-LTS)	March 2018	September 2018	Not Available	Indefinite
11 (LTS)	September 2018	September 2023	September 2026	Indefinite
12 (non-LTS)	March 2019	September 2019	Not Available	Indefinite
13 (non-LTS)	September 2019 ^{***}	March 2020	Not Available	Indefinite

<https://blogs.oracle.com/java/entry/java-se-support-roadmap-2019-05-06>

为了更快地迭代, Java 的更新从传统的以**特性驱动**的发布周期, 转变为以**时间驱动**的(6 个月为周期)发布模式——**每半年发布一个大版本, 每个季度发布一个中间特性版本**, 并且承诺不会跳票。通过这样的方式, 开发团队可以把一些关键特性尽早合并到 JDK 之中, 以快速得到开发者反馈, 在一定程度上避免出现像 Java 9 这样两次被迫延迟发布的窘况。

按照官方的说法，新的发布周期会严格遵循时间点，将于每年的3月份和9月份发布。所以 Java 11 的版本号是 18.9(LTS, long term support)。Oracle 直到2023年9月都会为 Java 11 提供技术支持，而补丁和安全警告等扩展支持将持续到2026年。

新的长期支持版本每三年发布一次，根据后续的发布计划，下一个长期支持版 Java 17 将于2021年发布。

4.2 版本升级的破坏性

模型	旧模型		新模型	
升级	Java主要版本	Java更新版本	Java发布列车	Java补丁
频率	每3年左右	每6个月一次	每6个月一次	每3个月一次
版本	6 - > 7 - > 8	8 - > 8u20 - > 8u40	11 - > 12 - > 13	11 - > 11.0.1 - > 11.0.2
语言变化	☑	X	☑	X
JVM更改	☑	X	☑	X
主要增强功能	☑	X	☑	X
添加了类/方法	☑	X	☑	X
删除了类/方法	X	X	☑	X
新的弃用	☑	X	☑	X
内部增强功能	☑	☑	☑	X
JDK工具更改	☑	☑	☑	X
Bug修复	☑	☑	☑	☑
安全补丁	☑	☑	☑	☑

Oracle 的官方观点认为：与 Java 7->8->9 相比，Java 9->10->11的升级和 8->8u20->8u40 更相似。

表格清楚地显示新模式下的 Java 版本发布都会包含许多变更，包括语言变更和 JVM 变更，这两者都会对 IDE、字节码库和框架产生重大影响。此外，不仅会新增其他 API，还会有 API 被删除（这在 Java 8 之前没有发生过）。

Oracle 的观点是，因为每个版本仅在前一个版本发布后的 6 个月推出，所以不会有太多新的“东西”，因此升级并不困难。虽然如此，但这不是重点。**重要的是升级是否有可能破坏代码。**很明显，从 11 -> 12 -> 13 开始，代码遭受破坏的可能性要大于 8 -> 8u20 -> 8u40。

11 -> 12 -> 13 与 8u20 -> 8u40 等这样的更新主要区别在于对字节码版本的更改以及对规范的更改，对字节码版本的更改往往特别具有破坏性，大多数框架都大量使用与每个字节码版本密切相关的 ASM 或 ByteBuddy 等库。而 8u20 -> 8u40 仍然使用相同的 Java SE 规范，具有所有相同的类和方法，不同于从 Java 12 移动到 13。

除此之外，Oracle 的另一个声明也十分值得我们关注。声明透露出的消息是，如果坚持使用 Java 11 并计划在下一个 LTS 版本(即 Java 17)发布时再进行升级，开发者可能会发现自己的项目代码无法通过编译。所以请记住，**Java 新的开发规则现在声明可以在一个版本中弃用某个 API 方法，并在下一个版本中删除它。**因此，一个方法可以在13中弃用并在15中删除。有人从11升级到17只会找到一个从未见过弃用的已删除API。

4.3 目前企业JDK版本使用统计

下图是开源中国 2018年09月统计的数据，样本512人



类比：

Java 1：



Java 8：



Java 11 :



Java 12 & 13 :



五、JDK与IDE的下载与安装

5.1 JDK 12的下载

Java 12版本对应的jdk

- 1 | jdk下载索引页 :
- 2 | <https://www.oracle.com/technetwork/java/javase/downloads/index.html>
- 3 | jdk12下载页 :
- 4 | <https://www.oracle.com/technetwork/java/javase/downloads/jdk12-downloads-5295953.html>

API文档 :

- 1 | <http://cr.openjdk.java.net/~iris/se/12/latestSpec/api/index.html>

安装完毕后 :

```
C:\Users\Administrator>java -version
java version "12.0.2" 2019-07-16
Java(TM) SE Runtime Environment (build 12.0.2+10)
Java HotSpot(TM) 64-Bit Server VM (build 12.0.2+10, mixed mode, sharing)
```

5.2 JDK 13的下载

Java 12版本对应的 jdk

- 1 | <https://www.oracle.com/technetwork/java/javase/downloads/jdk13-downloads-5672538.html>

API文档 :

- 1 | <http://cr.openjdk.java.net/~iris/se/13/latestSpec/api/index.html>

安装完毕后 :

```
C:\Users\Administrator>java -version
java version "13" 2019-09-17
Java(TM) SE Runtime Environment (build 13+33)
Java HotSpot(TM) 64-Bit Server VM (build 13+33, mixed mode, sharing)
```

5.3 IDE : IDEA的2019.2版的安装

- 1 | 官网下载地址 :
- 2 | <https://www.jetbrains.com/idea/whatsnew/>



2019.2 (Jul 24) ^

[Java](#)[Profiling Tools](#)[Services](#)[Performance](#)[Editor](#)[Appearance](#)[Gradle](#)[Maven](#)[Version Control](#)[Kotlin](#)[Groovy](#)

```
System.out.println("Web connection");
yield
switchExpressionCanReturnYield(int port)
notify()
notifyAll()
private enum PortType {
    HTTP, DATABASE, UNUSED, UNKNOWN, FTP, BUSY, SAFE
}
```

Java 13

IntelliJ IDEA is getting ready to welcome new Java 13 Preview features. The IDE provides support for updated Switch

Expressions and their new syntax: now if you need to return a value from a multi-line block in Java 13, you can use the yield keyword instead of break. We've also added support for text blocks, which allows you to embed longer multi-line blocks of text into your source code, for example, HTML or SQL. With

```
} else if (foo.
return "2";
} else if (foo.
return "3";
} else return "
```

Refactoring points

We've added a r
method with mu
prepare it for th
include multiple
statements. Whe
can be modified

六、Java 12新特性

美国当地时间 2019 年 3 月 19 日，也就是北京时间 20 号 Java 12 正式发布了！



- 1 JDK 12 is the open-source reference implementation of version 12 of the Java SE12 Platform as specified by by JSR 386 in the Java Community Process.
- 2
- 3 JDK 12 reached General Availability on 19 March 2019. Production-ready binaries under the GPL are available from Oracle; binaries from other vendors will follow shortly.
- 4 The features and schedule of this release were proposed and tracked via the JEP Process, as amended by the JEP 2.0 proposal. The release was produced using the JDK Release Process(JEP 3).

Features：总共有8个新的JEP(JDK Enhancement Proposals)。

- 1 <http://openjdk.java.net/projects/jdk/12/>

分别为：

```
1 189:Shenandoah:A Low-Pause-Time Garbage Collector(Experimental)
2 低暂停时间的GC
3 http://openjdk.java.net/jeps/189
4
5 230:Microbenchmark Suite
6 微基准测试套件
7 http://openjdk.java.net/jeps/230
8
9 325:Switch Expressions(Preview)
10 switch表达式
11 http://openjdk.java.net/jeps/325
12
13 334:JVM Constants API
14 JVM常量API
15 http://openjdk.java.net/jeps/334
16
17 340:One AArch64 Port,Not Two
18 只保留一个AArch64实现
19 http://openjdk.java.net/jeps/340
20
21 341:Default CDS Archives
22 默认类数据共享归档文件
23 http://openjdk.java.net/jeps/341
24
25 344:Abortable Mixed Collections for G1
26 可中止的G1 Mixed GC
27 http://openjdk.java.net/jeps/344
28
29 346:Promptly Return Unused Committed Memory from G1
30 G1及时返回未使用的已分配内存
31 http://openjdk.java.net/jeps/346
```

6.1 switch表达式 (预览)

6.1.1 传统switch的弊端

传统的switch声明语句(switch statement)在使用中有一些问题：

- 匹配是自上而下的，如果忘记写break, 后面的case语句不论匹配与否都会执行；
- 所有的case语句共用一个块范围，在不同的case语句定义的变量名不能重复；
- 不能在一个case里写多个执行结果一致的条件；
- 整个switch不能作为表达式返回值；

Java 12将会对switch声明语句进行扩展，可将其作为增强版的 switch 语句或称为 "switch 表达式"来写出更加简化的代码。

6.1.2 何为预览语言？

Switch 表达式也是作为预览语言功能的第一个语言改动被引入新版 Java 中来的，预览语言功能的想法是在 2018 年初被引入 Java 中的，**本质上讲，这是一种引入新特性的测试版的方法**。通过这种方式，能够根据用户反馈进行升级、更改，在极端情况下，如果没有被很好的接纳，则可以完全删除该功能。预览功能的关键在于它们没有被包含在 Java SE 规范中。

6.1.3 语法详解

扩展的 switch 语句，不仅可以作为语句（statement），还可以作为表达式（expression），并且两种写法都可以使用传统的 switch 语法，或者使用简化的“case L ->”模式匹配语法作用于不同范围并控制执行流。这些更改将简化日常编码工作，并为 switch 中的模式匹配（JEP 305）做好准备。

- 使用 Java 12 中 Switch 表达式的写法，省去了 break 语句，避免了因少写 break 而出错。
- 同时将多个 case 合并到一行，显得简洁、清晰也更加优雅的表达逻辑分支，其具体写法就是将之前的 case 语句表成了：case L ->，即如果条件匹配 case L，则执行标签右侧的代码，同时标签右侧的代码段只能是表达式、代码块或 throw 语句。
- 为了保持兼容性，case 条件语句中依然可以使用字符：，这时 fall-through 规则依然有效的，即不能省略原有的 break 语句，但是同一个 Switch 结构里不能混用 -> 和：，否则会有编译错误。并且简化后的 Switch 代码块中定义的局部变量，其作用域就限制在代码块中，而不是蔓延到整个 Switch 结构，也不用根据不同的判断条件来给变量赋值。

6.1.4 代码举例

举例1：

Java 12之前

```
1  /**
2   * @author shkstart
3   * @create 2019 下午 4:47
4   */
5  public class SwitchTest {
6      public static void main(String[] args) {
7          int numberOfLetters;
8          Fruit fruit = Fruit.APPLE;
9          switch (fruit) {
10             case PEAR:
11                 numberOfLetters = 4;
12                 break;
13             case APPLE:
14             case GRAPE:
15             case MANGO:
16                 numberOfLetters = 5;
17                 break;
18             case ORANGE:
19             case PAPAYA:
20                 numberOfLetters = 6;
21                 break;
22             default:
23                 throw new IllegalStateException("No Such Fruit:" + fruit);
24             }
25             System.out.println(numberOfLetters);
26         }
27     }
28 }
```

```

29 enum Fruit {
30     PEAR, APPLE, GRAPE, MANGO, ORANGE, PAPAYA;
31 }

```

如果有编码经验，你一定知道，switch 语句如果漏写了一个 break，那么逻辑往往就跑偏了，这种方式既繁琐，又容易出错。如果换成 switch 表达式，Pattern Matching 机制能够自然地保证只有单一路径会被执行：

Java 12

```

1  /**
2   * @author shkstart
3   * @create 2019 下午 10:38
4   */
5  public class SwitchTest1 {
6      public static void main(String[] args) {
7          Fruit fruit = Fruit.GRAPE;
8          switch(fruit){
9              case PEAR -> System.out.println(4);
10             case APPLE,MANGO,GRAPE -> System.out.println(5);
11             case ORANGE,PAPAYA -> System.out.println(6);
12             default -> throw new IllegalStateException("No Such Fruit:" + fruit);
13         };
14     }
15 }

```

更进一步，下面的表达式，为我们提供了优雅地表达特定场合计算逻辑的方式：

```

1  /**
2   * @author shkstart
3   * @create 2019 下午 10:44
4   */
5  public class SwitchTest2 {
6      public static void main(String[] args) {
7          Fruit fruit = Fruit.GRAPE;
8          int numberOfLetters = switch(fruit){
9              case PEAR -> 4;
10             case APPLE,MANGO,GRAPE -> 5;
11             case ORANGE,PAPAYA -> 6;
12             default -> throw new IllegalStateException("No Such Fruit:" + fruit);
13         };
14         System.out.println(numberOfLetters);
15     }
16 }

```

举例2：

Java 12之前：

```

1  public class SwitchTest {
2
3      public static void main(String[] args) {
4          Week day = Week.FRIDAY;

```

```

5      switch (day) {
6      case MONDAY:
7      case FRIDAY:
8      case SUNDAY:
9          System.out.println(6);
10         break;
11      case TUESDAY:
12          System.out.println(7);
13         break;
14      case THURSDAY:
15      case SATURDAY:
16          System.out.println(8);
17         break;
18      case WEDNESDAY:
19          System.out.println(9);
20         break;
21      default:
22          throw new IllegalStateException("what day is today?" + day);
23      }
24  }
25 }
26
27 enum Week {
28     MONDAY, TUESDAY, WEDNESDAY, THURSDAY, FRIDAY, SATURDAY, SUNDAY;
29 }

```

Java 12 :

```

1  /**
2   * @author shkstart
3   * @create 2019 下午 11:06
4   */
5  public class SwitchTest1 {
6
7      public static void main(String[] args) {
8          Week day = Week.FRIDAY;
9          switch (day) {
10             case MONDAY,FRIDAY, SUNDAY -> System.out.println(6);
11             case TUESDAY -> System.out.println(7);
12             case THURSDAY, SATURDAY -> System.out.println(8);
13             case WEDNESDAY -> System.out.println(9);
14             default -> throw new IllegalStateException("what day is today?" + day);
15         }
16     }
17 }
18

```

Java 12中更进一步：

```

1  /**
2   * @author shkstart
3   * @create 2019 下午 11:06

```

```

4  */
5  public class SwitchTest2 {
6      public static void main(String[] args) {
7          Week day = Week.FRIDAY;
8          int numLetters = switch (day) {
9              case MONDAY, FRIDAY, SUNDAY -> 6;
10             case TUESDAY -> 7;
11             case THURSDAY, SATURDAY -> 8;
12             case WEDNESDAY -> 9;
13             default -> throw new IllegalStateException("what day is today?" + day);
14         };
15     }
16 }

```

6.1.5 使用总结

这个语法如果做过Android开发的不会陌生，因为Kotlin 家的 when 表达式就是这么干的！

Switch Expressions 或者说起相关的 Pattern Matching 特性，为我们勾勒出了 Java 语法进化的一个趋势，将开发者从复杂繁琐的低层次抽象中逐渐解放出来，以更高层次更优雅的抽象，既降低代码量，又避免意外编程错误的出现，进而提高代码质量和开发效率。

6.1.6 展望

Java 11 以及之前版本中，Switch 表达式支持下面类型：byte、char、short、int、Byte、Character、Short、Integer、enum、String，在未来的某个 Java 版本有可能会允许支持 float、double 和 long（以及对应类型的包装类型）。

6.2 Shenandoah GC：低停顿时间的GC（预览）

6.2.1 背景和设计思路

Shenandoah 垃圾回收器是 Red Hat 在 2014 年宣布进行的一项垃圾收集器研究项目 Pauseless GC 的实现，旨在**针对 JVM 上的内存收回实现低停顿的需求**。该设计将与应用程序线程并发，通过交换 CPU 并发周期和空间以改善停顿时间，使得垃圾回收器执行线程能够在 Java 线程运行时进行堆压缩，并且标记和整理能够同时进行，因此避免了在大多数 JVM 垃圾收集器中所遇到的问题。

据 Red Hat 研发 Shenandoah 团队对外宣称，Shenandoah 垃圾回收器的暂停时间与堆大小无关，这意味着无论将堆设置为 200 MB 还是 200 GB，都将拥有一致的系统暂停时间，不过实际使用性能将取决于实际工作堆的大小和工作负载。

与其他 Pauseless GC 类似，**Shenandoah GC 主要目标是 99.9% 的暂停小于 10ms，暂停与堆大小无关等**。

这是一个实验性功能，不包含在默认（Oracle）的 OpenJDK 版本中。

6.2.2 补充：STW

Stop-the-World，简称 STW，指的是 GC 事件发生过程中，停止所有的应用程序线程的执行。就像警察办案，需要清场一样。

垃圾回收器的任务是识别和回收垃圾对象进行内存清理。为了让垃圾回收器可以正常且高效地执行，大部分情况下会要求系统进入一个停顿的状态。停顿的目的是终止所有应用程序的执行，只有这样，系统中才不会有新的垃圾产生，同时停顿**保证了系统状态在某一个瞬间的一致性**，也有益于垃圾回收器更好地标记垃圾对象。因此，在垃圾回收时，都会产生应用程序的停顿。停顿产生时整个应用程序会被暂停，没有任何响应，有点像卡死的感觉，这个停顿称为STW。

如果Stop-the-World 出现在新生代的Minor GC 中时，由于新生代的内存空间通常都比较小，所以暂停时间也在可接受的合理范围之内，不过一旦出现在老年代的Full GC 中时，程序的工作线程被暂停的时间将会更久。简单来说，**内存空间越大，执行Full GC 的时间就会越久，相对的工作线程被暂停的时间也会更长。**

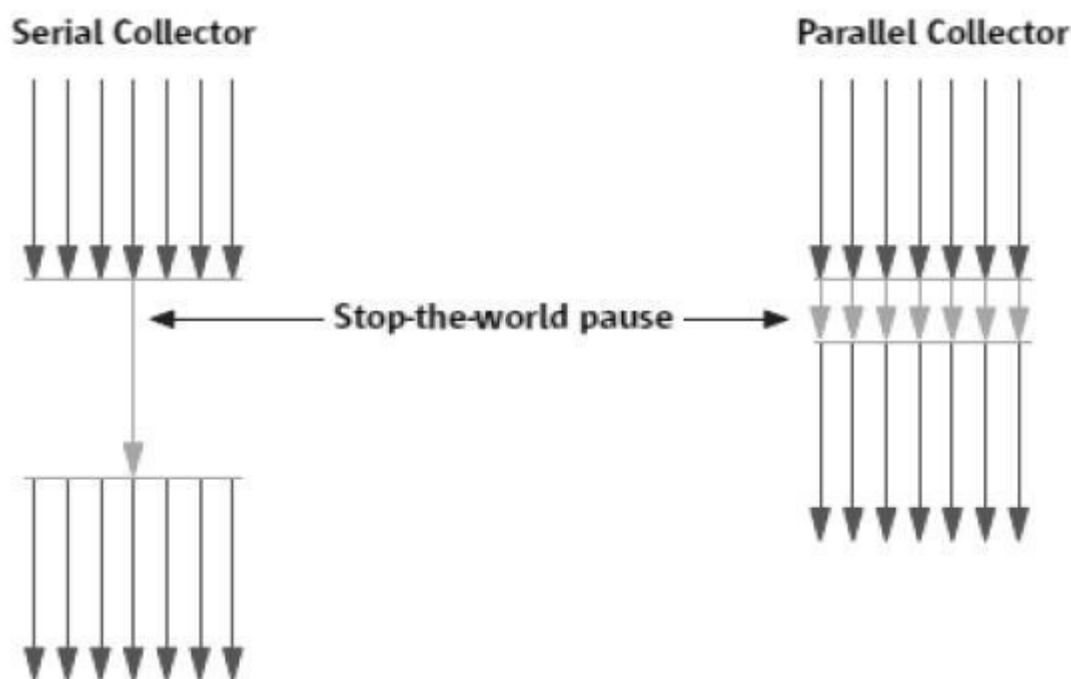
到目前为止，哪怕是G1 也不能完全避免Stop-the-world 情况发生，只能说垃圾回收器越来越优秀，回收效率越来越高，尽可能地缩短了暂停时间。

6.2.3 补充：垃圾收集器的分类

由于JDK 的版本处于高速迭代过程中，因此Java 发展至今已经衍生了众多的GC 版本。

从不同角度分析垃圾收集器，可以将GC 分为不同的类型。

- 按线程数分，可以分为串行垃圾回收器和并行垃圾回收器。



- 串行回收指的是在同一时间段内只允许一件事情发生，简单来说，当多个CPU 可用时，也只能有一个CPU 用于执行垃圾回收操作，并且在执行垃圾回收时，程序中的工作线程将会被暂停，当垃圾收集工作完成后才会恢复之前被暂停的工作线程，这就是串行回收。
- 和串行回收相反，并行收集可以运用多个CPU 同时执行垃圾回收，因此提升了应用的吞吐量，不过并行回收仍然与串行回收一样，采用独占式，使用了“ Stop-the-world ”机制和复制算法。
- 按照工作模式分，可以分为并发式回收器和独占式垃圾回收器。
 - 并发式垃圾回收器与应用程序线程交替工作，以尽可能减少应用程序的停顿时间。
 - 独占式垃圾回收器 (Stop the world)一旦运行，就停止应用程序中的其他所有线程，直到垃圾回收过程完全结束。
- 按碎片处理方式可分为压缩式垃圾回收器和非压缩式垃圾回收器。
 - 压缩式垃圾回收器会在回收完成后，对存活对象进行压缩整理，消除回收后的碎片。
 - 非压缩式的垃圾回收器不进行这步操作。

- 按工作的内存区间，又可分为年轻代垃圾回收器和老年代垃圾回收器。

6.2.4 补充：如何评估一款GC的性能

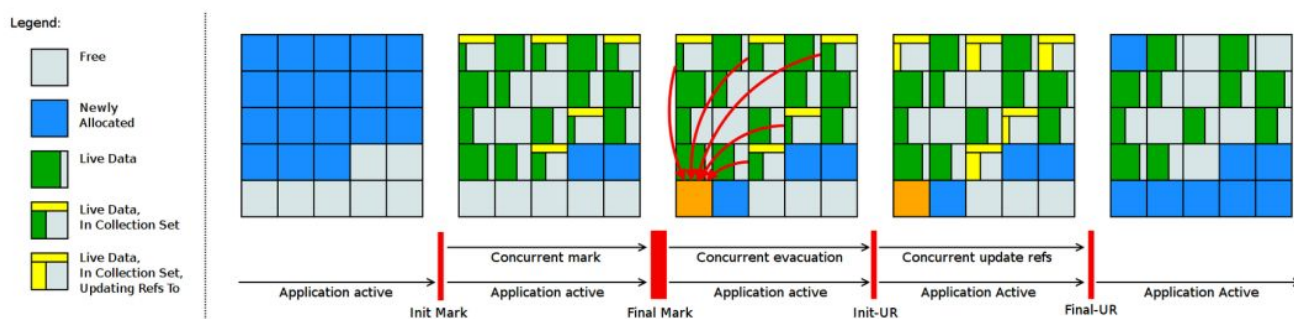
- 吞吐量：程序的运行时间（程序的运行时间 + 内存回收的时间）。
- 垃圾收集开销：吞吐量的补数，垃圾收集器所占时间与总时间的比例。
- 暂停时间：执行垃圾收集时，程序的工作线程被暂停的时间。
- 收集频率：相对于应用程序的执行，收集操作发生的频率。
- 堆空间：Java 堆区所占的内存大小。
- 快速：一个对象从诞生到被回收所经历的时间。

需要注意的是，垃圾收集器中吞吐量和低延迟这两个目标本身是相互矛盾的，因为如果选择以吞吐量优先，那么必然需要降低内存回收的执行频率，但是这样会导致GC 需要更长的暂停时间来执行内存回收。相反的，如果选择以低延迟优先为原则，那么为了降低每次执行内存回收时的暂停时间，也只能频繁地执行内存回收，但这又引起了年轻代内存的缩减和导致程序吞吐量的下降。

6.2.5 工作原理

从原理的角度，我们可以参考该项目官方的示意图，其内存结构与 G1 非常相似，都是将内存划分为类似棋盘的 region。整体流程与 G1 也是比较相似的，**最大的区别在于实现了并发的 疏散(Evacuation) 环节，引入的 Brooks Forwarding Pointer 技术使得 GC 在移动对象时，对象引用仍然可以访问。**

Shenandoah GC 工作周期如下所示：



- 1 上图对应工作周期如下：
- 2 1. Init Mark 启动并发标记 阶段
- 3 2. 并发标记遍历堆阶段
- 4 3. 并发标记完成阶段
- 5 4. 并发整理回收无活动区域阶段
- 6 5. 并发 Evacuation 整理内存区域阶段
- 7 6. Init Update Refs 更新引用初始化 阶段
- 8 7. 并发更新引用阶段
- 9 8. Final Update Refs 完成引用更新阶段
- 10 9. 并发回收无引用区域阶段

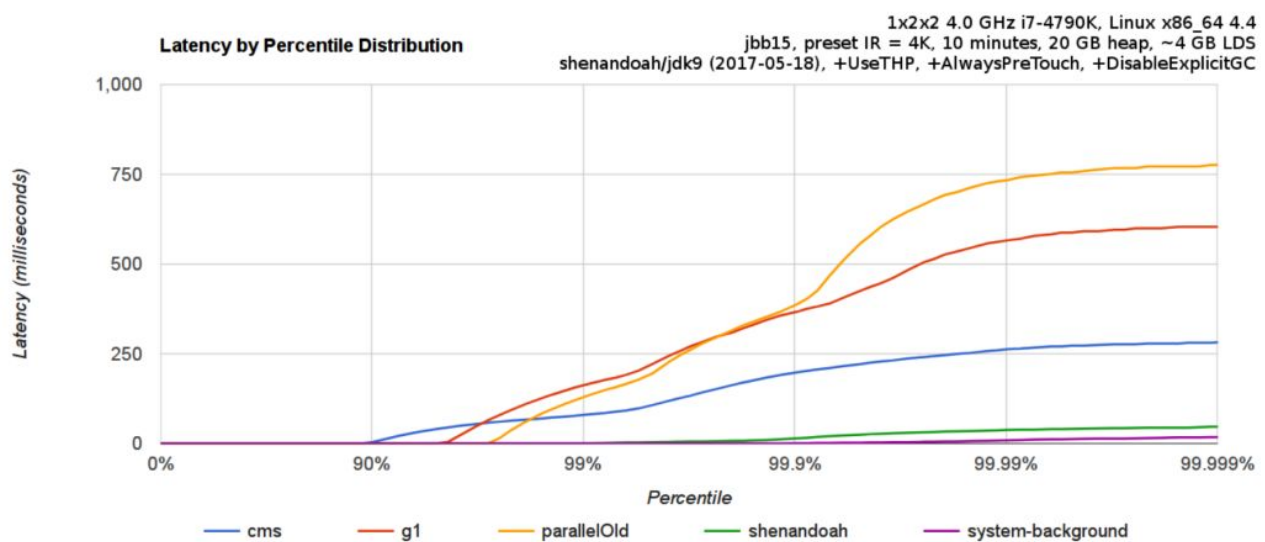
6.2.6 信息延展

也许了解 Shenandoah GC 的人比较少，业界声音比较响亮的是 Oracle 在 JDK11 中开源出来的 ZGC，或者商业版本的 Azul C4 (Continuously Concurrent Compacting Collector)。但是，至少目前我认为，其实际意义大于后两者，因为：

- 使用 ZGC 的最低门槛是升级到 JDK11，对很多团队来说，这种版本的跳跃并不是非常低成本的事情，更何况是尚不清楚 ZGC 在自身业务场景中的实际表现如何。
- 而 C4，毕竟是土豪们的选择，现实情况是，有多少公司连个几十块钱的 License 都不舍得。
- 而 Shenandoah GC 可是有稳定的 JDK8u 版本发布的。据了解，已经有个别公司在 HBase 等高实时性产品中实践许久。
- ZGC也是面向low-pause-time的垃圾收集器，不过ZGC是基于colored pointers来实现，而Shenandoah GC是基于brooks pointers来实现。

需要了解，**不是唯有 GC 停顿可能导致常规应用程序响应时间比较长**。具有较长的 GC 停顿时间会导致系统响应慢的问题，但响应时间慢并非一定是 GC 停顿时间长导致的，**队列延迟、网络延迟、其他依赖服务延迟和操作提供调度程序抖动等都可能响应变慢**。使用 Shenandoah 时需要全面了解系统运行情况，综合分析系统响应时间。下面是 jbb15 benchmark 中，Shenandoah GC 相对于其他主流 GC 的表现。

各种 GC 工作负载对比：



GC 暂停相比于 CMS 等选择有数量级程度的提高，对于 GC 暂停非常敏感的场景，价值还是很明显的，能够在 SLA 层面有显著提高。当然，**这种对于低延迟的保证，也是以消耗 CPU 等计算资源为代价的**，实际吞吐量表现也不是非常明朗，需要看企业的实际场景需求，并不是一个一劳永逸的解决方案。

下面推荐几个配置或调试 Shenandoah 的 JVM 参数:

```

1  -XX:+AlwaysPreTouch : 使用所有可用的内存分页，减少系统运行停顿，为避免运行时性能损失。
2
3  -Xmx == -Xmsv : 设置初始堆大小与最大值一致，可以减轻伸缩堆大小带来的压力，与 AlwaysPreTouch 参数配合使用，在启动时提交所有内存，避免在最终使用中出现系统停顿。
4
5  -XX:+ UseTransparentHugePages : 能够大大提高大堆的性能，同时建议在 Linux 上使用时将 /sys/kernel/mm/transparent_hugepage/enabled 和 /sys/kernel/mm/transparent_hugepage/defragv 设置为: madvise，同时与 AlwaysPreTouch 一起使用时，init 和 shutdownv 速度会更快，因为它将使用更大的页面进行预处理。
6
7  -XX:+UseNUMA : 虽然 Shenandoah 尚未明确支持 NUMA (Non-Uniform Memory Access)，但最好启用此功能以在多插槽主机上启用 NUMA 交错。与 AlwaysPreTouch 相结合，它提供了比默认配置更好的性能。
8
9  -XX:+DisableExplicitGC : 忽略代码中的 System.gc() 调用。当用户在代码中调用 System.gc() 时会强制 Shenandoah 执行 STW Full GC，应禁用它以防止执行此操作，另外还可以使用 -XX:+ExplicitGCInvokesConcurrent，在调用 System.gc() 时执行 CMS GC 而不是 Full GC，建议在有 System.gc() 调用的情况下使用。
10
11 不过目前 Shenandoah 垃圾回收器还被标记为实验项目，如果要使用Shenandoah GC需要编译时--with-jvm-features选项带有shenandoahgc，然后启动时使用参数
12  -XX:+UnlockExperimentalVMOptions -XX:+UseShenandoahGC

```

6.3 JVM 常量 API

Java 12 中引入 JVM 常量 API，用来更容易地对关键类文件 (key class-file) 和运行时构件 (artefact) 的名义描述 (nominal description) 进行建模，特别是对那些从常量池加载的常量，这是一项非常技术性的变化，能够以更简单、标准的方式处理可加载常量。

具体来说就是java.base模块新增了java.lang.constant包（而非 java.lang.invoke.constant）。包中定义了一系列基于值的符号引用（JVMS 5.1）类型，它们能够描述每种可加载常量。

```

1  官方api链接地址：
2  http://cr.openjdk.java.net/~iris/se/12/latestSpec/api/java.base/java/lang/constant/package-summary.html

```

```

1  Java SE > Java SE Specifications > Java Virtual Machine Specification下的第5章：
2  Chapter 5. Loading, Linking, and Initializing
3
4  https://docs.oracle.com/javase/specs/jvms/se7/html/jvms-5.html

```

引入了ConstantDesc接口(ClassDesc、MethodTypeDesc、MethodHandleDesc这几个接口直接继承了ConstantDesc接口)以及Constable接口；ConstantDesc接口定义了resolveConstantDesc方法，Constable接口定义了describeConstable方法；String、Integer、Long、Float、Double均实现了这两个接口，而EnumDesc实现了ConstantDesc接口

Interface Summary

Interface	Description
ClassDesc	A nominal descriptor for a Class constant.
Constable	Represents a type which is <i>constable</i> .
ConstantDesc	A nominal descriptor for a loadable constant value, as defined in JVM 4.4.
DirectMethodHandleDesc	A nominal descriptor for a direct MethodHandle.
MethodHandleDesc	A nominal descriptor for a MethodHandle constant.
MethodTypeDesc	A nominal descriptor for a MethodType constant.

Class Summary

Class	Description
ConstantDescs	Predefined values of nominal descriptor for common constants, including descriptors for primitive class types and other common platform types, and descriptors for method handles for standard bootstrap methods.
DynamicCallSiteDesc	A nominal descriptor for an invokedynamic call site.
DynamicConstantDesc<T>	A nominal descriptor for a dynamic constant (one described in the constant pool with Constant_Dynamic_info.)

符号引用以纯 nominal 形式描述可加载常量，与类加载或可访问性上下文区分开。有些类可以作为自己的符号引用（例如 String）。而对于可链接常量，另外定义了一系列符号引用类型，具体包括：ClassDesc (Class 的可加载常量标称描述符)，MethodTypeDesc (方法类型常量标称描述符)，MethodHandleDesc (方法句柄常量标称描述符) 和 DynamicConstantDesc (动态常量标称描述符)，它们包含描述这些常量的 nominal 信息。**此 API 对于操作类和方法的工具很有帮助。**

6.3.1 String 实现了 Constable 接口：

```
1 public final class String implements java.io.Serializable, Comparable<String>,
   CharSequence, Constable, ConstantDesc {
```

java.lang.constant.Constable接口定义了抽象方法：

```
1 public interface Constable {
2     Optional<? extends ConstantDesc> describeConstable();
3 }
```

Java 12 String 的实现源码：

```
1 @Override
2 public Optional<String> describeConstable() {
3     return Optional.of(this);
4 }
```

很简单，其实就是调用 Optional.of 方法返回一个 Optional 类型，Optional不懂的可以参考Java 8的新特性

6.3.2 String#describeConstable和resolveConstantDesc

一个非常有趣的方法来自新引入的接口java.lang.constant.Constable - 它用于标记constable类型，这意味着这类型的值是常量，可以在JVM 4.4常量池中定义。

```
1 Java SE > Java SE Specifications > Java Virtual Machine Specification下的第4章 :
2 Chapter 4. The class File Format
3
4 https://docs.oracle.com/javase/specs/jvms/se7/html/jvms-4.html
```

String的源码：

```
1 /**
2  * Returns an {@link Optional} containing the nominal descriptor for this
3  * instance, which is the instance itself.
4  *
5  * @return an {@link Optional} describing the {@linkplain String} instance
6  * @since 12
7  */
8 @Override
9 public Optional<String> describeConstable() {
10     return Optional.of(this);
11 }
12
13
14 /**
15  * Resolves this instance as a {@link ConstantDesc}, the result of which is
16  * the instance itself.
17  *
18  * @param lookup ignored
19  * @return the {@linkplain String} instance
20  * @since 12
21  */
22 @Override
23 public String resolveConstantDesc(MethodHandles.Lookup lookup) {
24     return this;
25 }
```

举例：

```
1 private static void testDescribeConstable() {
2     System.out.println("====test java 12 describeConstable====");
3     String name = "尚硅谷Java高级工程师";
4     Optional<String> optional = name.describeConstable();
5     System.out.println(optional.get());
6 }
```

结果输出：

```
1 =====test java 12 describeConstable=====
2 尚硅谷Java高级工程师
```

6.4 微基准测试套件

6.4.1 何为JMH?

JMH，即Java Microbenchmark Harness，是专门用于代码微基准测试的工具套件。何谓Micro Benchmark呢？简单的来说就是基于方法层面的基准测试，精度可以达到微秒级。当你定位到热点方法，希望进一步优化方法性能的时候，就可以使用JMH对优化的结果进行量化的分析。

6.4.2 JMH比较典型的应用场景：

- 想准确的知道某个方法需要执行多长时间，以及执行时间和输入之间的相关性；
- 对比接口不同实现在给定条件下的吞吐量；
- 查看多少百分比的请求在多长时间完成；

6.4.3 JMH的使用

要使用JMH，首先需要准备好Maven环境，JMH的源代码以及官方提供的Sample就是使用Maven进行项目管理的，github上也有使用gradle的例子可自行搜索参考。使用mvn命令行创建一个JMH工程：

```
1 mvn archetype:generate \  
2     -DinteractiveMode=false \  
3     -DarchetypeGroupId=org.openjdk.jmh \  
4     -DarchetypeArtifactId=jmh-java-benchmark-archetype \  
5     -DgroupId=co.speedar.infra \  
6     -DartifactId=jmh-test \  
7     -Dversion=1.0
```

如果要在现有Maven项目中使用JMH，只需要把生成出来的两个依赖以及shade插件拷贝到项目的pom中即可：

```
1 <dependency>  
2     <groupId>org.openjdk.jmh</groupId>  
3     <artifactId>jmh-core</artifactId>  
4     <version>0.7.1</version>  
5 </dependency>  
6 <dependency>  
7     <groupId>org.openjdk.jmh</groupId>  
8     <artifactId>jmh-generator-annprocess</artifactId>  
9     <version>0.7.1</version>  
10    <scope>provided</scope>  
11 </dependency>  
12 ...  
13 <plugin>  
14     <groupId>org.apache.maven.plugins</groupId>  
15     <artifactId>maven-shade-plugin</artifactId>  
16     <version>2.0</version>  
17     <executions>  
18         <execution>  
19             <phase>package</phase>  
20             <goals>  
21                 <goal>shade</goal>  
22             </goals>  
23             <configuration>  
24                 <finalName>microbenchmarks</finalName>  
25                 <transformers>
```

```

26         <transformer
implementation="org.apache.maven.plugins.shade.resource.ManifestResourceTransformer">
27             <mainClass>org.openjdk.jmh.Main</mainClass>
28         </transformer>
29     </transformers>
30 </configuration>
31 </execution>
32 </executions>
33 </plugin>

```

6.4.4 新特性的说明

Java 12 中添加一套新的基本的微基准测试套件（microbenchmarks suite），此功能为JDK源代码添加了一套微基准测试（大约100个），**简化了现有微基准测试的运行和新基准测试的创建过程**。使开发人员可以轻松运行现有的微基准测试并创建新的基准测试，其目标在于提供一个稳定且优化过的基准。它基于Java Microbenchmark Harness（JMH），可以轻松测试JDK性能，支持JMH更新。

微基准套件与JDK源代码位于同一个目录中，并且在构建后将生成单个jar文件。但它是一个单独的项目，在支持构建期间不会执行，以方便开发人员和其他对构建微基准套件不感兴趣的人在构建时花费比较少的构建时间。

要构建微基准套件，用户需要运行命令：make build-microbenchmark，类似的命令还有：make test TEST="micro:java.lang.invoke" 将使用默认设置运行 java.lang.invoke 相关的微基准测试。

6.5 只保留一个 AArch64 实现

6.5.1 现状

当前Java 11 及之前版本JDK中存在两个64位ARM端口。这些文件的主要来源位于 `src/hotspot/cpu/arm` 和 `open/src/hotspot/cpu/aarch64` 目录中。尽管两个端口都产生了 `aarch64` 实现，我们将前者（由Oracle贡献）称为 `arm64`，将后者称为 `aarch64`。

6.5.2 新特性

Java 12 中将删除由 Oracle 提供的 `arm64`端口相关的所有源码，即删除目录 `open/src/hotspot/cpu/arm` 中关于 64-bit 的这套实现，只保留其中有关 32-bit ARM端口的实现，余下目录的 `open/src/hotspot/cpu/aarch64` 代码部分就成了 AArch64 的默认实现。

6.5.3 目的

这将使开发贡献者将他们的精力集中在单个 64 位 ARM 实现上，并消除维护两套实现所需的重复工作。

6.6 默认生成类数据共享(CDS)归档文件

6.6.1 概述

我们知道在同一个物理机 / 虚拟机上启动多个JVM时，如果每个虚拟机都单独装载自己需要的所有类，启动成本和内存占用是比较高的。所以Java团队引入了类数据共享机制（Class Data Sharing，简称 CDS）的概念，通过把一些核心类在每个JVM间共享，每个JVM只需要装载自己的应用类即可。好处是：启动时间减少了，另外核心类是共享的，所以JVM的内存占用也减少了。

6.6.2 历史版本

- JDK5引入了Class-Data Sharing可以用于多个JVM共享class，提升启动速度，最早只支持system classes及serial GC。
- JDK9对其进行扩展以支持application classes及其他GC算法。
- java10的新特性[JP 310: Application Class-Data Sharing](#)扩展了JDK5引入的Class-Data Sharing，支持application的Class-Data Sharing并开源出来(以前是commercial feature)
 - CDS 只能作用于 BootClassLoader 加载的类，不能作用于 AppClassLoader 或者自定义的 ClassLoader 加载的类。在 Java 10 中，则将 CDS 扩展为 AppCDS，顾名思义，AppCDS 不止能够作用于 BootClassLoader了，AppClassLoader 和自定义的 ClassLoader 也都能够起作用，大大加大了 CDS 的适用范围。也就说开发自定义的类也可以装载给多个JVM共享了。
- JDK11将-Xshare:off改为默认-Xshare:auto，以更加方便使用CDS特性

6.6.3 迭代效果

可以说，自 Java 8 以来，在基本 CDS 功能上进行了许多增强、改进，启用 CDS 后应用的启动时间和内存占用量显著减少。使用 Java 11 早期版本在 64 位 Linux 平台上运行 HelloWorld 进行测试，测试结果显示启动时间缩短有 32 %，同时在其他 64 位平台上，也有类似或更高的启动性能提升。

6.6.4 Java12新特性

JDK 12之前，想要利用CDS的用户，即使仅使用JDK中提供的默认类列表，也必须 `java -Xshare:dump` 作为额外的步骤来运行。

Java 12 针对 64 位平台下的 JDK 构建过程进行了增强改进，使其默认生成类数据共享（CDS）归档，以进一步达到**改进应用程序的启动时间的目的**，同时也**避免了需要手动运行：`java -Xshare:dump` 的需要**，修改后的 JDK 将在 `${JAVA_HOME}/lib/server` 目录中生成一份名为classes.jsa的默认archive文件(大概有18M)方便大家使用。

当然如果需要，也可以添加其他 GC 参数，来调整堆大小等，以获得更优的内存分布情况，同时用户也可以像之前一样创建自定义的 CDS 存档文件。

6.7 可中断的 G1 Mixed GC

简言之，当 G1 垃圾回收器的回收超过暂停时间的目标，则能中止垃圾回收过程。

G1是一个垃圾收集器，设计用于具有大量内存的多处理器机器。由于它提高了性能效率，G1垃圾收集器最终将取代CMS垃圾收集器。

该垃圾收集器设计的主要目标之一是满足用户设置的预期的 JVM 停顿时间。

G1 采用一个高级分析引擎来选择在收集期间要处理的工作量，此选择过程的结果是一组称为 GC 回收集（collection set(CSet)）的区域。一旦收集器确定了 GC 回收集 并且 GC 回收、整理工作已经开始，这个过程是without stopping的，即 G1 收集器必须完成收集集合的所有区域中的所有活动对象之后才能停止；但是如果收集器选择过大的 GC 回收集，此时的STW时间会过长超出目标pause time。

这种情况在mixed collections时候比较明显。这个特性启动了一个机制，当选择了一个比较大的collection set，Java 12 中将把 GC 回收集（混合收集集合）拆分为mandatory（必需或强制）及optional两部分(当完成mandatory的部分，如果还有剩余时间则会去处理optional部分)来将mixed collections从without stopping变为abortable，以更好满足指定pause time的目标。

- 其中必需处理的部分包括 G1 垃圾收集器不能递增处理的 GC 回收集的部分（如：年轻代），同时也可以包含老年代以提高处理效率。
- 将 GC 回收集拆分为必需和可选部分时，垃圾收集过程优先处理必需部分。同时，需要为可选 GC 回收集部分维护一些其他数据，这会产生轻微的 CPU 开销，但小于 1 % 的变化，同时在 G1 回收器处理 GC 回收集期间，本机内存使用率也可能会增加，使用上述情况只适用于包含可选 GC 回收部分的 GC 混合回收集合。
- 在 G1 垃圾回收器完成收集需要必需回收的部分之后，如果还有时间的话，便开始收集可选的部分。但是粗粒度的处理，可选部分的处理粒度取决于剩余的时间，一次只能处理可选部分的一个子集区域。在完成可选收集部分的收集后，G1 垃圾回收器可以根据剩余时间决定是否停止收集。如果在处理完必需处理的部分后，剩余时间不足，总时间花销接近预期时间，G1 垃圾回收器也可以中止可选部分的回收以达到满足预期停顿时间的目标。

6.8 增强G1，自动返回未用堆内存给操作系统

6.8.1 概述

上面介绍了 Java 12 中增强了 G1 垃圾收集器关于混合收集集合的处理策略，这节主要介绍在 Java 12 中同时也对 G1 垃圾回收器进行了改进，**使其能够在空闲时自动将 Java 堆内存返还给操作系统**，这也是 Java 12 中的另外一项重大改进。

目前 Java 11 版本中包含的 G1 垃圾收集器暂时无法及时将已提交的 Java 堆内存返还给操作系统。为什么呢？G1 目前只有在 full GC 或者 concurrent cycle（并发处理周期）的时候才会归还内存，由于这两个场景都是 G1 极力避免的，因此在大多数场景下可能不会及时归还 committed Java heap memory 给操作系统。除非有外部强制执行。

在使用云平台的容器环境中，这种不利之处特别明显。即使在虚拟机不活动，但如果仍然使用其分配的内存资源，哪怕是其中的一小部分，G1 回收器也仍将保留所有已分配的 Java 堆内存。而这将导致用户需要始终为所有资源付费，哪怕是实际并未用到，而云提供商也无法充分利用其硬件。**如果在此期间虚拟机能够检测到 Java 堆内存的实际使用情况，并在利用空闲时间自动将 Java 堆内存返还，则两者都将受益。**

6.8.2 具体操作

为了尽可能的向操作系统返回空闲内存，**G1 垃圾收集器将在应用程序不活动期间定期生成或持续循环检查整体 Java 堆使用情况**，以便 G1 垃圾收集器能够更及时的将 Java 堆中不使用内存部分返还给操作系统。对于长时间处于空闲状态的应用程序，此项改进将使 JVM 的内存利用率更加高效。

而在用户控制下，可以可选地执行 Full GC，以使返回的内存量最大化。

JDK12 的这个特性新增了两个参数分别是 G1PeriodicGCInterval 及 G1PeriodicGCSystemLoadThreshold，设置为 0 的话，表示禁用。如果应用程序为非活动状态，在下面两种情况任何一个描述下，G1 回收器会触发定期垃圾收集：

- 自上次垃圾回收完成以来已超过 G1PeriodicGCInterval (milliseconds)，并且此时没有正在进行的垃圾回收任务。如果 G1PeriodicGCInterval 值为零表示禁用快速回收内存的定期垃圾收集。
- 应用所在主机系统上执行方法 getloadavg()，默认一分钟内系统返回的平均负载值低于 G1PeriodicGCSystemLoadThreshold 指定的阈值，则触发 full GC 或者 concurrent GC (如果开启 G1PeriodicGCInvokesConcurrent)，GC 之后 Java heap size 会被重写调整，然后多余的内存将会归还给操作系统。如果 G1PeriodicGCSystemLoadThreshold 值为零，则此条件不生效。

如果不满足上述条件中的任何一个，则取消当期的定期垃圾回收。等一个 G1PeriodicGCInterval 时间周期后，将重新考虑是否执行定期垃圾回收。

G1 定期垃圾收集的类型根据 `G1PeriodicGCInvokesConcurrent` 参数的值确定：如果设置值了，G1 垃圾回收器将继续上一个或者启动一个新并发周期；如果没有设置值，则 G1 回收器将执行一个 Full GC。在每次一次 GC 回收末尾，G1 回收器将调整当前的 Java 堆大小，此时便有可能会将未使用内存返还给操作系统。新的 Java 堆内存大小根据现有配置确定，具体包括下列配置：`-XX:MinHeapFreeRatio`、`-XX:MaxHeapFreeRatio`、`-Xms`、`-Xmx`。

默认情况下，G1 回收器在定期垃圾回收期间新启动或继续上一轮并发周期，将最大限度地减少应用程序的中断。如果定期垃圾收集严重影响程序执行，则需要考虑整个系统 CPU 负载，或让用户禁用定期垃圾收集。

6.9 其他解读

上面列出的是大方面的特性，除此之外还有一些api的更新及废弃，主要见[JDK 12 Release Notes](#)，这里举几个例子。

6.9.1 增加项：支持unicode 11

```
1  JDK 12版本包括对Unicode 11.0.0的支持。在发布支持Unicode 10.0.0的JDK 11之后，Unicode 11.0.0引
   入了以下JDK 12中包含的新功能：
2  684 new characters
3  11 new blocks
4  7 new scripts.
5  其中：
6  684个新字符，包含以下重要内容：
7      66个表情符号字符 (66 emoji characters)
8      Copyleft符号 (Copyleft symbol)
9      评级系统的半星 (Half stars for rating systems)
10     额外的占星符号 (Additional astrological symbols)
11     象棋中国象棋符号 (Xiangqi Chinese chess symbols)
12 7个新脚本：
13     Hanifi Rohingya
14     Old Sogdian
15     Sogdian
16     Dogra
17     Gunjala Gondi
18     Makasar
19     Medefaidrin
20 11个新块，包括上面列出的新脚本的7个块和以下现有脚本的4个块：
21     格鲁吉亚扩展 (Georgian Extended)
22     玛雅数字 (Mayan Numerals)
23     印度Siyag数字 (Indic Siyag Numbers)
24     国际象棋符号 (Chess Symbols)
```

6.9.2 增加项：支持压缩数字格式化

`NumberFormat` 添加了对以紧凑形式格式化数字的支持。紧凑数字格式是指以简短或人类可读形式表示的数字。例如，在`en_US`语言环境中，1000可以格式化为“1K”，1000000可以格式化为“1M”，具体取决于指定的样式 `NumberFormat.Style`。

```

1  @Test
2  public void testCompactNumberFormat(){
3      var cnf = NumberFormat.getCompactNumberInstance(Locale.CHINA,
4          NumberFormat.Style.SHORT);
5      System.out.println(cnf.format(1_0000));
6      System.out.println(cnf.format(1_9200));
7      System.out.println(cnf.format(1_000_000));
8      System.out.println(cnf.format(1L << 30));
9      System.out.println(cnf.format(1L << 40));
10     System.out.println(cnf.format(1L << 50));
11 }

```

输出

```

1  1万
2  2万
3  100万
4  11亿
5  1兆
6  1126兆

```

6.9.3 增加项：String新增方法

1. String#transform(Function)

JDK-8203442引入的一个小方法，它提供的函数作为输入提供给特定的String实例，并返回该函数返回的输出。

```

1  var result = "foo".transform(input -> input + " bar");
2  System.out.println(result); // foo bar

```

或者

```

1  var result = "foo"
2      .transform(input -> input + " bar")
3      .transform(String::toUpperCase)
4  System.out.println(result); // FOO BAR

```

对应的源码：

```

1  /**
2   * This method allows the application of a function to {@code this}
3   * string. The function should expect a single String argument
4   * and produce an {@code R} result.
5   * <p>
6   * Any exception thrown by {@code f()} will be propagated to the
7   * caller.
8   *
9   * @param f    functional interface to a apply
10  *
11  * @param <R>  class of the result
12  *

```

```

13  * @return    the result of applying the function to this string
14  *
15  * @see java.util.function.Function
16  *
17  * @since 12
18  */
19  public <R> R transform(Function<? super String, ? extends R> f) {
20      return f.apply(this);
21  }

```

传入一个函数式接口 Function，接受一个值，返回一个值，参考：Java 8 新特性之函数式接口。

在某种情况下，该方法应该被称为map()。

举例：

```

1  private static void testTransform() {
2      System.out.println("====test java 12 transform====");
3      List<String> list1 = List.of("Java", " Python", " C++ ");
4      List<String> list2 = new ArrayList<>();
5      list1.forEach(element -> list2.add(element.transform(String::strip)
6                                          .transform(String::toUpperCase)
7                                          .transform((e) -> "Hi," + e))
8      );
9      list2.forEach(System.out::println);
10 }

```

结果输出：

```

1  =====test java 12 transform=====
2  Hi, JAVA
3  Hi, PYTHON
4  Hi, C++

```

示例是对一个字符串连续转换了三遍，代码很简单。如果使用Java 8的Stream特性，可以如下实现：

```

1  private static void testTransform1() {
2      System.out.println("====test before java 12 =====");
3      List<String> list1 = List.of("Java ", " Python", " C++ ");
4
5      Stream<String> stringStream = list1.stream().map(element ->
6      element.strip()).map(String::toUpperCase).map(element -> "Hello," + element);
7      List<String> list2 = stringStream.collect(Collectors.toList());
8      list2.forEach(System.out::println);
9  }

```

2. String#indent

该方法允许我们调整String实例的缩进。

举例：

```

1 private static void testIndent() {
2     System.out.println("====test java 12 indent====");
3     String result = "Java\n Python\nC++".indent(3);
4     System.out.println(result);
5 }

```

结果输出：

```

1 =====test java 12 indent=====
2     Java
3     Python
4     C++

```

换行符 \n 后向前缩进 n 个空格，为 0 或负数不缩进。

以下是 indent 的核心源码：

```

1 /**
2  * Adjusts the indentation of each line of this string based on the value of
3  * {@code n}, and normalizes line termination characters.
4  * <p>
5  * This string is conceptually separated into lines using
6  * {@link String#lines()}. Each line is then adjusted as described below
7  * and then suffixed with a line feed {@code "\n"} (U+000A). The resulting
8  * lines are then concatenated and returned.
9  * ...略...
10  *
11  * @since 12
12  */
13 public String indent(int n) {
14     if (isEmpty()) {
15         return "";
16     }
17     Stream<String> stream = lines();
18     if (n > 0) {
19         final String spaces = " ".repeat(n);
20         stream = stream.map(s -> spaces + s);
21     } else if (n == Integer.MIN_VALUE) {
22         stream = stream.map(s -> s.stripLeading());
23     } else if (n < 0) {
24         stream = stream.map(s -> s.substring(Math.min(-n,
25 s.indexOfNonwhitespace())));
26     }
27     return stream.collect(Collectors.joining("\n", "", "\n"));
28 }

```

其实就是调用了 lines() 方法来创建一个 Stream，然后再往前拼接指定数量的空格。

6.9.4 新增项：Files新增mismatch方法

```

1 @Test

```

```

2      public void testFilesMismatch() throws IOException {
3          FileWriter filewriter = new FileWriter("tmp\\a.txt");
4          filewriter.write("a");
5          filewriter.write("b");
6          filewriter.write("c");
7          filewriter.close();
8
9          FileWriter filewriterB = new FileWriter("tmp\\b.txt");
10         filewriterB.write("a");
11         filewriterB.write("1");
12         filewriterB.write("c");
13         filewriterB.close();
14
15         System.out.println(Files.mismatch(Path.of("tmp/a.txt"), Path.of("tmp/b.txt")));
16     }

```

6.9.5 新增项：其他

- Collectors新增teeing方法用于聚合两个downstream的结果
- CompletionStage新增exceptionallyAsync、exceptionallyComposeAsync方法，允许方法体在异步线程执行，同时新增了exceptionallyCompose方法支持在exceptionally的时候构建新的CompletionStage。
- ZGC: Concurrent Class Unloading
 - ZGC在JDK11的时候还不支持class unloading，JDK12对ZGC支持了Concurrent Class Unloading，默认是开启，使用-XX:-ClassUnloading可以禁用
- 新增-XX:+ExtensiveErrorReports
 - -XX:+ExtensiveErrorReports可以用于在jvm crash的时候收集更多的报告信息到hs_err.log文件中，product builds中默认是关闭的，要开启的话，需要自己添加-XX:+ExtensiveErrorReports参数
- 新增安全相关的改进
 - 支持java.security.manager系统属性，当设置为disallow的时候，则不使用SecurityManager以提升性能，如果此时调用System.setSecurityManager则会抛出UnsupportedOperationExceptionkeytool新增-groupname选项允许在生成key pair的时候指定一个named group新增PKCS12 KeyStore配置属性用于自定义PKCS12 keystores的生成Java Flight Recorder新增了security-related的event支持ChaCha20 and Poly1305 TLS Cipher Suites

6.9.6 移除项

- 移除com.sun.awt.SecurityWarnin；
- 移除FileInputStream、FileOutputStream、- Java.util.ZipFile/Inflator/Deflator的finalize方法；
- 移除GTE CyberTrust Global Root；
- 移除javac的-source, -target对6及1.6的支持，同时移除--release选项；

6.9.7 废弃项

- 废弃的API列表见deprecated-list
- 废弃-XX:+/-MonitorInUseLists选项
- 废弃Default Keytool的-keyalg值

6.10 开发者如何看待 Java 12



此外，不少开发者对 Raw String Literals 特性情有独钟，该特性类似于 JavaScript ES6 语法中的模板字符串，使用它基本可以告别丑陋的字符串拼接。特性详见：

1 | <http://openjdk.java.net/jeps/326>

该特性原计划于 JDK 12 发布，可惜最后还是被取消了，详见：

1 | <http://mail.openjdk.java.net/pipermail/jdk-dev/2018-December/002402.html>

可能是因为业界呼声太高，最近委员会又把这个特性拿出来重新讨论了：

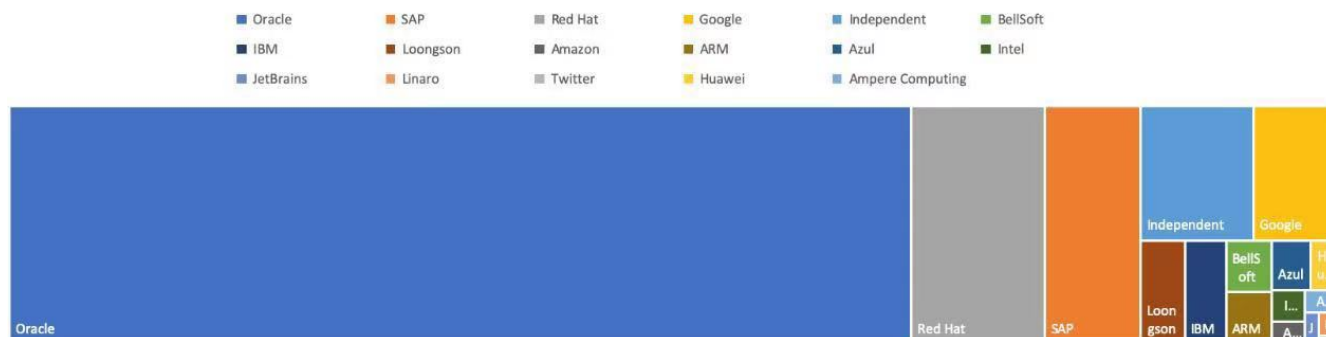
1 | <https://mail.openjdk.java.net/pipermail/amber-spec-experts/2019-January/000931.html>

七、Java 13新特性

2019年9月17日，国际知名的OpenJDK开源社区发布了Java编程语言环境的最新版本OpenJDK13。



Issues fixed in JDK 13



根据Oracle的统计信息，如上图所示，在所有为社区JDK 13有代码贡献的公司中，排名前五的为：Oracle、Red Hat、SAP、Google和龙芯。龙芯位列全球第5，全国第1，为社区贡献了几十个Patch。

```

1 Features : 总共有5个新的JEP(JDK Enhancement Proposals):
2 http://openjdk.java.net/projects/jdk/13/
3
4 各个 build 的更新说明可以查看:
5 https://jdk.java.net/13/release-notes

```

Features

```

1 350:Dynamic CDS Archives
2 动态CDS档案
3
4 351:ZGC: Uncommit Unused Memory
5 ZGC:取消使用未使用的内存
6
7 353:Reimplement the Legacy Socket API
8 重新实现旧版套接字API
9
10 354:Switch Expressions (Preview)
11 switch表达式 (预览)
12
13 355:Text Blocks (Preview)
14 文字块 (预览)

```

7.1 switch表达式 (预览)

在JDK 12中引入了Switch表达式作为预览特性。JDK 13提出了第二个switch表达式预览。JEP 354修改了这个特性，它引入了**yield语句，用于返回值**。这意味着，switch表达式(返回值)应该使用yield, switch语句(不返回值)应该使用break。

在JDK 12中有一个，但是要进行一个更改：要从switch表达式中生成一个值break，要删除with value语句以支持yield声明。目的是扩展，switch以便它可以用作语句或表达式，因此两个表单既可以使用case ...: 带有连贯符号的传统标签，也可以使用新case ... -> 标签，而不需要通过，还有一个新的语句用于从switch表达式中产生值。这些更改将简化编码并为**模式匹配**做好准备。

在以前，我们想要在switch中返回内容，还是比较麻烦的，一般语法如下：

```

1  @Test
2  public void testSwitch1(){
3      String x = "3";
4      int i;
5      switch (x) {
6          case "1":
7              i=1;
8              break;
9          case "2":
10             i=2;
11             break;
12         default:
13             i = x.length();
14             break;
15     }
16     System.out.println(i);
17 }

```

在JDK13中使用以下语法：

```

1  @Test
2  public void testSwitch2(){
3      String x = "3";
4      int i = switch (x) {
5          case "1" -> 1;
6          case "2" -> 2;
7          default -> {
8              yield 3;
9          }
10     };
11     System.out.println(i);
12 }

```

或者

```

1  @Test
2  public void testSwitch3() {
3      String x = "3";
4      int i = switch (x) {
5          case "1":
6              yield 1;
7          case "2":
8              yield 2;
9          default:
10             yield 3;
11     };
12     System.out.println(i);
13 }

```

在这之后，switch中就多了一个关键字用于跳出switch块了，那就是yield，他用于返回一个值。和return的区别在于：**return会直接跳出当前循环或者方法，而yield只会跳出当前switch块。**

7.2 文本块（预览）

在JDK 12中引入了Raw String Literals特性，但在发布之前就放弃了。这个JEP与引入多行字符串文字（text block）在意义上是类似的。

这条新特性跟 Kotlin 里的文本块是类似的。

现实问题

在Java中，通常需要使用String类型表达HTML，XML，SQL或JSON等格式的字符串，在进行字符串赋值时需要进行转义和连接操作，然后才能编译该代码，这种表达方式难以阅读并且难以维护。

文本块就是指多行字符串，例如一段格式化后的xml、json等。而有了文本块以后，用户不需要转义，Java能自动搞定。因此，**文本块将提高Java程序的可读性和可写性。**

目标

- 简化跨越多行的字符串，避免对换行等特殊字符进行转义，简化编写Java程序。
- 增强Java程序中字符串的可读性。

举例

会被自动转义，如有一段以下字符串：

```
1 <html>
2   <body>
3     <p>Hello, 尚硅谷</p>
4   </body>
5 </html>
```

将其复制到Java的字符串中，会展示成以下内容：

```
1 "<html>\n" +
2 "  <body>\n" +
3 "    <p>Hello, 尚硅谷</p>\n" +
4 "  </body>\n" +
5 "</html>\n";
```

即被自动进行了转义，这样的字符串看起来不是很直观，在JDK 13中，就可以使用以下语法了：

```
1 ""
2 <html>
3   <body>
4     <p>Hello, world</p>
5   </body>
6 </html>
7 "";
```

使用""作为文本块的开始符和结束符，在其中就可以放置多行的字符串，不需要进行任何转义。看起来就十分清爽了。

如常见的SQL语句：

```

1 select employee_id,last_name,salary,department_id
2 from employees
3 where department_id in (40,50,60)
4 order by department_id asc

```

原有方式：

```

1 String query = "select employee_id,last_name,salary,department_id\n" +
2               "from employees\n" +
3               "where department_id in (40,50,60)\n" +
4               "order by department_id asc";

```

使用新特性：

```

1 String newQuery = ""
2     select employee_id,last_name,salary,department_id
3     from employees
4     where department_id in (40,50,60)
5     order by department_id asc
6 "";

```

具体使用

1.基本使用

- **文本块**是Java语言中的一种新文字。它可以用来表示任何字符串，并且提供更大的表现力和更少的复杂性。
- **文本块**由零个或多个字符组成，由开始和结束分隔符括起来。
 - 开始分隔符是由三个双引号字符（`"""`），后面可以跟零个或多个空格，最终以行终止符结束。文本块内容以开始分隔符的行终止符后的第一个字符开始。
 - 结束分隔符也是由三个双引号字符（`"""`）表示，文本块内容以结束分隔符的第一个双引号之前的最后一个字符结束。
- **文本块**中的内容可以直接使用`"`，`,`，但不是必需的。
- **文本块**中的内容可以直接包括行终止符。允许在文本块中使用`\n`，但不是必需的。例如，文本块：

```

1 """
2 line1
3 line2
4 line3
5 """

```

相当于：

```

1 "line1\nline2\nline3\n"

```

或者一个连接的字符串：

```
1 "line1\n" +
2 "line2\n" +
3 "line3\n"
```

如果字符串末尾不需要行终止符，则结束分隔符可以放在最后一行内容上。例如：

```
1 ""
2 line1
3 line2
4 line3""
```

相当于

```
1 "line1\nline2\nline3"
```

文本块可以表示空字符串，但不建议这样做，因为它需要两行源代码：

```
1 String empty = ""
2 "";
```

以下示例是错误格式的文本块：

```
1 String a = """"; // 开始分隔符后没有行终止符
2 String b = "" "" ""; // 开始分隔符后没有行终止符
3 String c = ""
4 "; // 没有结束分隔符
5 String d = ""
6 abc \ def
7 """; // 含有未转义的反斜线（请参阅下面的转义处理）
```

在运行时，**文本块将被实例化为String的实例，就像字符串一样**。从文本块派生的String实例与从字符串派生的实例是无法区分的。**具有相同内容的两个文本块将引用相同的String实例，就像字符串一样**。

此外，

2. 编译器在编译时会删除掉这些多余的空格。

下面这段代码中，我们用.来表示我们代码中的的空格，而这些位置的空格就是多余的。

```
1 String html = ""
2 .....<html>
3 .....    <body>
4 .....        <p>Hello, world</p>
5 .....    </body>
6 .....</html>
7 ....."";
```

多余的空格还会出现在每一行的结尾，特别是当你从其他地方复制过来时，更容易出现这种情况，比如下面的代码：

```

1 String html = ""
2 .....<html>...
3 .....    <body>
4 .....        <p>Hello, world</p>....
5 .....    </body>.
6 .....</html>...
7 ....."";

```

这些多余的空格对于程序员来说是看不到的，但是他又是实际存在的，所以**如果编译器不做处理，可能会导致程序员看到的两个文本块内容是一样的，但是这两个文本块却因为存在这种多余的空格而导致差异，比如哈希值不相等。**

3. 转义字符

允许开发人员使用 `\n`、`\f` 和 `\r` 来进行字符串的垂直格式化，使用 `\b` 和 `\t` 进行水平格式化。比如下面的代码是合法的：

```

1 String html = ""
2     <html>\n
3     <body>\n
4         <p>Hello, world</p>\n
5     </body>\n
6 </html>\n
7 "";

```

请注意，在文本块内自由使用"是合法的。例如：

```

1 String story = ""
2     "when I use a word," Humpty Dumpty said,
3     in rather a scornful tone, "it means just what I
4     choose it to mean - neither more nor less."
5     "The question is," said Alice, "whether you
6     can make words mean so many different things."
7     "The question is," said Humpty Dumpty,
8     "which is to be master - that's all."
9     "";

```

但是，三个"字符的序列需要进行转义至少一个"以避免模仿结束分隔符：

```

1 String code =
2     ""
3     String text = \"\"\"
4         A text block inside a text block
5     \"\"\";
6     \"\"\";

```

4. 文本块连接

可以在任何可以使用字符串的地方使用文本块。例如，文本块和字符串可以相互连接：


```

1 String code = "public void print(Object o) {" +
2             ""
3             System.out.println(Objects.toString(o));
4             }
5             """;

```

但是，涉及文本块的连接可能变得相当笨重。以下面文本块为基础：

```

1 String code = ""
2             public void print(Object o) {
3             System.out.println(Objects.toString(o));
4             }
5             """;

```

假设我们想把上面的Object改为来自某一变量，我们可能会这么写：

```

1 String code = ""
2             public void print("" + type + ""
3             o) {
4             System.out.println(Objects.toString(o));
5             }
6             """;

```

可以发现这种写法可读性是非常差的，更简洁的替代方法是使用String :: replace或String :: format，比如：

```

1 String code = ""
2             public void print($type o) {
3             System.out.println(Objects.toString(o));
4             }
5             ""$.replace("$type", type);

```

```

1 String code = String.format(""
2             public void print(%s o) {
3             System.out.println(Objects.toString(o));
4             }
5             "", type);

```

另一个方法是使用String :: formatted，这是一个新方法，比如：

```

1 String source = ""
2             public void print(%s object) {
3             System.out.println(Objects.toString(object));
4             }
5             ""$.formatted(type);

```

7.3 动态CDS档案（动态类数据共享归档）

CDS，是java 12的特性了，可以让不同 Java 进程之间共享一份类元数据，减少内存占用，它还能加快应用的启动速度。而JDK13的这个特性**支持在Java application执行之后进行动态archive**。存档类将包括默认的基础层CDS存档中不存在的所有已加载的应用程序和库类。也就是说，在Java 13中再使用AppCDS的时候，就不再需要这么复杂了。

该提案处于目标阶段，旨在提高AppCDS的可用性，并消除用户进行试运行以创建每个应用程序的类列表的需要。

使用示例：

```
1 # JVM退出时动态创建共享归档文件：导出jsa
2 java -XX:ArchiveClassesAtExit=hello.jsa -cp hello.jar Hello
3
4 # 用动态创建的共享归档文件运行应用：使用jsa
5 java -XX:SharedArchiveFile=hello.jsa -cp hello.jar Hello
6
```

7.4 ZGC:取消使用未使用的内存

7.4.1 G1和Shenandoah

JVM的GC释放的内存会还给操作系统吗？

GC后的内存如何处置，其实是取决于不同的垃圾回收器。因为把内存还给OS，意味着要调整JVM的堆大小，这个过程是比较耗费资源的。

- Java12的[346: Promptly Return Unused Committed Memory from G1](#)新增了两个参数分别是G1PeriodicGCInterval及G1PeriodicGCSystemLoadThreshold用于GC之后重新调整Java heap size，然后将多余的内存归还给操作系统
- Java12的[189: Shenandoah: A Low-Pause-Time Garbage Collector \(Experimental\)](#)拥有参数-XX:ShenandoahUncommitDelay=来指定ZPage的page cache的失效时间，然后归还内存

HotSpot的G1和Shenandoah这两个GC已经提供了这种能力，并且对某些用户来说，非常有用。因此，Java13则给ZGC新增归还unused heap memory给操作系统的特性。

7.4.2 ZGC的使用背景

在JDK 11中，Java引入了ZGC，这是一款可伸缩的低延迟垃圾收集器，但是当时只是实验性的。号称不管你开了多大的堆内存，它都能保证在 10 毫秒内释放 JVM，不让它停顿在那。但是，当时的设计是它不能把内存归还给操作系统。对于比较关心内存占用的应用来说，肯定希望进程不要占用过多的内存空间了，所以这次增加了这个特性。

JDK 11

JDK 11 is the open-source reference implementation of version 11 of the Java SE Platform as specified by JSR 384 in the Java Community Process.

JDK 11 reached General Availability on 25 September 2018. Production-ready binaries under the GPL are available from Oracle; binaries from other vendors will follow shortly.

The features and schedule of this release were proposed and tracked via the JEP Process, as amended by the JEP 2.0 proposal. The release was produced using the JDK Release Process (JEP 3).

Features

- 181: Nest-Based Access Control
- 309: Dynamic Class-File Constants
- 315: Improve Aarch64 Intrinsics
- 318: Epsilon: A No-Op Garbage Collector
- 320: Remove the Java EE and CORBA Modules
- 321: HTTP Client (Standard)
- 323: Local-Variable Syntax for Lambda Parameters
- 324: Key Agreement with Curve25519 and Curve448
- 327: Unicode 10
- 328: Flight Recorder
- 329: ChaCha20 and Poly1305 Cryptographic Algorithms
- 330: Launch Single-File Source-Code Programs
- 331: Low-Overhead Heap Profiling
- 332: Transport Layer Security (TLS) 1.3
- 333: ZGC: A Scalable Low-Latency Garbage Collector (Experimental)
- 335: Deprecate the Nashorn JavaScript Engine

在Java 13中，JEP 351再次对ZGC做了增强，将没有使用的堆内存归还给操作系统。ZGC当前不能把内存归还给操作系统，即使是那些很久都没有使用的内存，也只进不出。这种行为并不是对任何应用和环境都是友好的，尤其是那些内存占用敏感的服务，例如：

1. 按需付费使用的容器环境；
2. 应用程序可能长时间闲置，并且和很多其他应用共享和竞争资源的环境；
3. 应用程序在执行期间有非常不同的堆空间需求，例如，可能在启动的时候所需的堆比稳定运行的时候需要更多的堆内存。

7.4.3 使用细节

ZGC的堆由若干个Region组成，每个Region被称之为**ZPage**。每个Zpage与数量可变的已提交内存相关联。当ZGC压缩堆的时候，ZPage就会释放，然后进入page cache，即**ZPageCache**。这些在page cache中的ZPage集合就表示没有使用部分的堆，这部分内存**应该**被归还给操作系统。回收内存可以简单的通过从page cache中逐出若干个选好的ZPage来实现，由于page cache是以LRU（Least recently used，最近最少使用）顺序保存ZPage的，并且按照尺寸（小，中，大）进行隔离，因此逐出ZPage机制和回收内存相对简单了很多，**主要挑战是设计关于何时从page cache中逐出ZPage的策略。**

一个简单的策略就是设定一个超时或者延迟值，表示ZPage被驱逐前，能在page cache中驻留多长时间。这个超时时间会有一个合理的默认值，也可以通过JVM参数覆盖它。Shenandoah GC用了一个类型的策略，默认超时时间是5分钟，可以通过参数-XX:ShenandoahUncommitDelay = milliseconds覆盖默认值。

像上面这样的策略可能会运作得相当好。但是，用户还可以设想更复杂的策略：**不需要添加任何新的命令行选项**。例如，基于GC频率或某些其他数据找到合适超时值的启发式算法。**JDK13将使用哪种具体策略目前尚未确定**。可能最初只提供一个简单的超时策略，使用-XX:ZUncommitDelay = seconds选项，以后的版本会添加更复杂、更智能的策略（如果可以的话）。

uncommit能力默认是开启的，但是无论指定何种策略，ZGC都不能把堆内存降低于Xms。这就意味着，如果Xmx和Xms相等的话，这个能力就失效了。-XX:-ZUncommit这个参数也能让这个内存管理能力失效。

7.5 重新实现旧版套接字API

7.5.1 现有问题

重新实现了古老的 Socket 接口。现在已有的 java.net.Socket 和 java.net.ServerSocket 以及它们的实现类，都可以回溯到JDK 1.0 时代了。

- 它们的实现是混合了 Java 和 C 的代码的，维护和调试都很痛苦。
- 实现类还使用了线程栈作为 I/O 的缓冲，导致在某些情况下还需要增加线程栈的大小。
- 支持异步关闭，此操作是通过使用一个本地的数据结构来实现的，这种方式这些年也带来了潜在的不稳定性和跨平台移植问题。该实现还存在几个并发问题，需要彻底解决。

在未来的网络世界，要快速响应，不能阻塞本地方法线程，当前的实现不适合使用了。

7.5.2 新的实现类

全新实现的 NioSocketImpl 来替换JDK1.0的PlainSocketImpl。

- 它便于维护和调试，与 NewI/O (NIO) 使用相同的 JDK 内部结构，因此不需要使用系统本地代码。
- 它与现有的缓冲区缓存机制集成在一起，这样就不需要为 I/O 使用线程栈。
- 它使用 java.util.concurrent 锁，而不是 synchronized 同步方法，增强了并发能力。
- 新的实现是Java 13中的默认实现，但是旧的实现还没有删除，可以通过设置系统属性 jdk.net.usePlainSocketImpl来切换到旧版本。

7.5.3 代码说明

运行一个实例化Socket和ServerSocket的类将显示这个调试输出。这是默认的(新的)。

```
1  Module java.base
2  Package java.net
3  Class SocketImpl
4  public abstract class SocketImpl implements SocketOptions {
5      private static final boolean USE_PLAIN_SOCKET_IMPL = usePlainSocketImpl();
6
7      private static boolean usePlainSocketImpl() {
8          PrivilegedAction<String> pa = () ->
9              NetProperties.get("jdk.net.usePlainSocketImpl");
10         String s = AccessController.doPrivileged(pa);
11         return (s != null) && !s.equalsIgnoreCase("false");
12     }
13
14     /**
15      * Creates an instance of platform's SocketImpl
16      */
17     @SuppressWarnings("unchecked")
```

```

17     static <S extends SocketImpl & PlatformSocketImpl> S
    createPlatformSocketImpl(boolean server) {
18         if (USE_PLAINSOCKETIMPL) {
19             return (S) new PlainSocketImpl(server);
20         } else {
21             return (S) new NioSocketImpl(server);
22         }
23     }
24 }

```

SocketImpl的USE_PLAINSOCKETIMPL取决于usePlainSocketImpl方法，而它会从NetProperties读取dk.net.usePlainSocketImpl配置，如果不为null且不为false，则usePlainSocketImpl方法返回true；createPlatformSocketImpl会根据USE_PLAINSOCKETIMPL来创建PlainSocketImpl或者NioSocketImpl。

7.6 其他解读

上面列出的是大方面的特性，除此之外还有一些api的更新及废弃，主要见JDK 13 Release Notes，这里举几个例子。

<https://jdk.java.net/13/release-notes>

7.6.1 增加项

- 添加FileSystems.newFileSystem(Path, Map<String, ?>) Method
- 新的java.nio.ByteBuffer Bulk get/put Methods Transfer Bytes Without Regard to Buffer Position
- 支持Unicode 12.1
- 添加-XX:SoftMaxHeapSize Flag，目前仅仅对ZGC起作用
- ZGC的最大heap大小增大到16TB

7.6.2 移除项

- 移除awt.toolkit System Property
- 移除Runtime Trace Methods
- 移除-XX:+AggressiveOpts
- 移除Two Comodo Root CA Certificates、Two DocuSign Root CA Certificates
- 移除内部的com.sun.net.ssl包

7.6.3 废弃项

- 废弃-Xverify:none及-noverify
- 废弃rmic Tool并准备移除
- 废弃javax.security.cert并准备移除

7.6.4 已知问题

- 不再支持Windows 2019 Core Server
- 使用ZIP File System (zipfs) Provider来更新包含Uncompressed Entries的ZIP或JAR可能造成文件损坏

7.6.5 其他事项

- GraphicsEnvironment.getCenterPoint()及getMaximumWindowBounds()已跨平台统一
- 增强了JAR Manifest的Class-Path属性处理

- 针对Negatively Sized Argument , StringBuffer(CharSequence)及StringBuilder(CharSequence)会抛出NegativeArraySizeException
- linux的默认进程启动机制已经使用posix_spawn
- Lookup.unreflectSetter(Field)针对static final fields会抛出IllegalAccessException
- 使用了java.net.Socket.setSocketImplFactory及java.net.ServerSocket.setSocketFactory方法的要注意，要求客户端及服务端要一致，不能一端使用自定义的factory一端使用默认的factory
- SocketImpl的supportedOptions, getOption及setOption方法的默认实现发生了变化，默认的supportedOptions返回空，而默认的getOption,及setOption方法抛出UnsupportedOperationException
- JNI NewDirectByteBuffer创建的Direct Buffer为java.nio.ByteOrder.BIG_ENDIAN
- Base64.Encoder及Base64.Decoder可能抛出OutOfMemoryError
- 改进了Serial GC Young pause time report
- 改进了MaxRAM及UseCompressedOops参数的行为

7.7 小结

以上，就是JDK13中包含的主要特性。

- 语法层面，改进了Switch Expressions，新增了Text Blocks，二者皆处于Preview状态；
- API层面主要使用NioSocketImpl来替换JDK1.0的PlainSocketImpl
- GC层面则改进了ZGC，以支持Uncommit Unused Memory

而且，JDK13并不是LTS（长期支持）版本，如果你正在使用Java 8（LTS）或者Java 11（LTS），暂时可以不必升级到Java 13。

这些年很多Java粉丝已经讨厌写那些冗长难看的代码了，有些转投到高效的Python门下，有的转用Kotlin，有的去了新兴的Go那边。不过，Java大叔凭着其广阔的领土，活跃高效的运转机构，以及其开放改革的心，还会有不少的粉丝追随他的。

八、采用新版本Java的注意事项

在采用新版本Java之前必须考虑的一些注意事项/风险。

注意1：被新版本系列“绑架”

如果采用了Java 12并使用新的语言特性或新的API，这意味着实际上你已将项目绑定到Java的新版本系列。接下来你必须采用Java 13, 14, 15, 16和17。

使用了新版本，每个版本的使用寿命为六个月，并且在发布后仅七个月就过时了。这是因为每个版本只有在六个月内提供安全补丁，发布后1个月的第一个补丁和发布后4个月的第二个补丁。7个月后，下一组安全补丁会发布，但旧版本不能获取更新。

因此，你要判断自身的开发流程是否允许升级Java版本，一个月的时间窗口方面会不会太狭窄？或者你是否愿意在安全基线以下的Java版本上运行？

注意2：升级的“绊脚石”

实际使用中有很多阻止我们升级Java的因素，下面列出一些常见的：

- **开发资源不足**：你的团队可能会非常忙碌或规模太小，你能保证两年后从Java 15升级到16的开发时间吗？
- **构建工具和IDE**：使用的IDE是否会在发布当天支持每个新版本？Maven? Gradle呢？如果不是，你有后备计划吗？请记住，你只有1个月的时间来完成升级、测试并将其发布到生产环境中。此外还包括Checkstyle，JaCoCo，PMD，SpotBugs等等其他工具。

- **依赖关系**：你的依赖关系是否都准备好用于每个新版本？请记住，它不仅仅是直接依赖项，而是技术堆栈中的所有内容。字节码操作库尤其受到影响，例如 ByteBuddy 和 ASM。
- **框架**：这是另一种依赖，但是一个大而重要的依赖。在一个月的狭窄时间窗口内，Spring 会每六个月发布一个新版本吗？Jakarta EE(以前的 Java EE)会吗？如果它们不这样做会怎么样？

现在，任何阻挡者的传统方法都是等待：在开始升级之前等待6到12个月，以便为工具，库和框架提供修复任何错误的机会。

注意3：云 / 托管 / 部署

你是否可以控制代码在生产环境中的运行位置和方式？例如，如果你在 AWS (Amazon Web Service) Lambda 中运行代码，则无法控制。AWS Lambda 没有采用 Java 9或10，甚至没有采用 Java 11。所以除非 AWS 提供公共保证以支持每个新的 Java 版本，否则根本无法采用 Java 12。（我的工作假设是AWS Lambda仅支持主要的LTS版本，由[Amazon Corretto JDK公告支持](#)。）

如何托管你的 CI 系统？Jenkins, Travis, Circle, Shippable, GitLab 会快速更新吗？如果不是，你会怎么做？

注意4：为采用新版本进行规划

如果正在考虑采用新版本的 Java，建议你准备一份现在所依赖的所有内容的清单，或者可能在未来3年内会依赖的。**你需要保证该列表中的所有内容都能正常工作，并与新版本一起升级，或者如果该依赖项不再更新，请制定好计划。**以某位互联网开发者为例，他列的清单如下：

- Amazon AWS
- Eclipse
- IntelliJ
- Travis CI
- Shippable CI
- Maven
- Maven 插件(compile, jar, source, javadoc等)
- Checkstyle, 以及相关的 IDE 插件和 maven 插件
- JaCoCo, 以及相关的 IDE 插件和 maven 插件
- PMD 和相关的 maven 插件
- SpotBugs 和相关的 maven 插件
- OSGi bundle metadata tool
- Bytecode 工具(Byte buddy / ASM etc)
- 超过 100 个 jar 包依赖项

说了这么多，当然不是鼓励大家不进行升级，新语言特性带来的好处以及性能增强会让开发者受益，但升级背后的风险也应该考虑进去。

注意5：其他第三方厂商的声明

Spring 框架已经在视频中表达了对 Java 12 的策略。关键部分是：

- 1 “Java 8 和 11 作为 LTS 版本会持续获得我们的正式支持，对于过渡版本，我们也会尽最大努力支持，但它们不会获得正式的生产环境支持。如果你升级到 Java 11，我们非常愿意和你合作，因为长期支持版本才是我们关注的重心，对于 Java 12 及更高版本我们会尽最大的努力。”

作为典型软件供应商的一个例子，Liferay 声明如下：

- 1 Liferay 已决定不会对 JDK 的每个主要版本进行认证。我们将选择遵循 Oracle 的主导并仅认证标记为 LTS 的版本。
— Liferay 博客

个人的想法：

- 1 想象一下汽车制造商的类似行为：
- 2
- 3 - 每6个月重新设计和发布一次汽车
- 4 - 从2018年开始，每三年只提供一次完整的保修
- 5 - 如果客户购买2019型号并且在6个月内出现问题，他们必须等待并购买2020修复模型
- 6 - 2020型号是电动的，但你的城镇的基础设施还不支持电充等设备支持，更不用说座椅已经改变并导致腰痛
- 7 - 不用担心，购买2020.3型号！

九、写在最后

9.1 再谈JDK版本更新

Oracle Java SE Support Roadmap ^{*†}				
Release	GA Date	Premier Support Until	Extended Support Until	Sustaining Support
6	December 2006	December 2015	December 2018	Indefinite
7	July 2011	July 2019	July 2022 ^{*****}	Indefinite
8 ^{**}	March 2014	March 2022	March 2025	Indefinite
9 (non-LTS)	September 2017	March 2018	Not Available	Indefinite
10 (non-LTS)	March 2018	September 2018	Not Available	Indefinite
11 (LTS)	September 2018	September 2023	September 2026	Indefinite
12 (non-LTS)	March 2019	September 2019	Not Available	Indefinite
13 (non-LTS)	September 2019 ^{***}	March 2020	Not Available	Indefinite

<https://blogs.oracle.com/java/entry/java-se-support-roadmap-10050>

在 Java 9 之前，当一个版本被宣布为首选版本，存在一个“培育”（bedded-in）新 GA 版本的重叠期。在此期间，上一个版本将会继续进行免费更新。为确保新旧版本间的干净切换，即便旧版本已不再是首选版本，通常也会继续维护 12 个月以上。但是随着 Java 版本发布更改为遵循严格的时间表后，事实上宣告了传统的免费支持期将寿终正寝。

也许不会有太多公司直接选择 JDK12、JDK 13，但个别的生产实践并不遥远。比如，实际工作场景中，利用 JDK 12 的 Abortable Mixed Collections for G1，解决了 HDFS 在特定场景中 G1 Evacuation 时间过长的困扰，虽然最后团队选择将其 backport 到了自己的 JDK11 版本中，但如果没有快速交付的预览版 JDK12，也不会如此快速的得到结论。

而对另一个问题，目前看是非常成功的，**解开了 Java/JVM 演进的许多枷锁，至关重要的是，OpenJDK 的权力中心，正在转移到开发社区和开发者手中。**在新的模式中，既可以利用 LTS 满足企业长期可靠支持的需求，也可以满足各种开发者对于新特性迭代的诉求。你可能注意到了 Switch Expressions 被打上了预览（Preview）的标签，Shenandoah GC 则是实验（Experimental）特性，这些都是以往的发布周期下不大现实的，因为用 2-3 年的最小间隔粒度来实验一个特性，基本是不现实的。

可以预计，JDK8 在未来的一段时间仍将是主流，我们已经注意到 Amazon、Alibaba、Redhat、AdoptOpenJDK 等厂商或社区，**纷纷发布了自己的 JDK8 等产品，开始竞赛长期支持版本 JDK 的主导权**，这是非常好的迹象，反映了主流厂商对于 Java 的投资力度增大。

是否会带来 Java/JVM 的碎片化呢？多少会发生一些，但从目前的合作模式来看，OpenJDK 仍然是合作的中心，主导这 Java 历史版本维护和未来的演进路线。

9.2 Oracle 撒手，OpenJDK 继续向前

Java 8 是目前使用率最高的一个 Java 版本，发布于 2014 年，而 Oracle 对 Java 8 的官方支持时间持续到 2020 年 12 月，之后将不再为个人桌面用户提供 Oracle JDK 8 的修复更新；在 2019 年 1 月之后，不再提供免费的商业版本更新，届时想要继续获得 Oracle 的商业支持和维护，需付费订阅。

近日，Oracle 的销售代表发出的一封邮件引起了热议，该邮件称“**Java 8 的非公开可用的关键补丁更新**”将于 **2019 年 4 月 16 日发布，拥有有效许可证的客户才可以享用。邮件继续称，如果没有安装这些更新，可能导致“你的服务器和桌面环境暴露且易受攻击。”**

但在许多 Java 用户看来，这封邮件像是一种敲诈勒索或恐吓策略。

This is an important Java functionality and security notice for the 4/16 Critical Release.

Hope this finds you well. I am reaching out to make sure you are aware that the first quarterly non-publicly available, critical patch update for Java 8 will be released April 16th. Any non-oracle applications and servers running Java Version 8 or later will require a license in order to continue receiving patches and updates, beyond the release. Without proper licensing in place, patching and updating will not be available, possibly leaving your server and desktop environment exposed and vulnerable.

I want to make sure you have the resources and information you need in this transition.

If this is something you feel needs to be addressed, please let me know and we can set something up accordingly pending availability.

虽然 Oracle 官方选择了不再支持，但 Java 社区却把担子接了下来。红帽 Java 平台团队的首席工程师 Andrew Haley 曾表示，红帽计划在 2023 年之前继续提供对 OpenJDK 8 的支持：

“在我看来，这算比较正常的。几年前，OpenJDK 6 更新 (jdk6u) 项目被 Oracle 放弃，我接管了它，然后 OpenJDK 7 也发生了同样的事情。最后，Azul 的 Andrew Brygin 接管了 OpenJDK 6。由来自多个组织成员组成的 OpenJDK Vulnerability Group 就重要的安全问题进行协作。**在广大的 OpenJDK 社区和我的团队 (Red Hat) 的帮助下，我们定期为关键 bug 和安全漏洞提供更新。我觉得这样的过程同样适用于 OpenJDK 8 和下一个长期支持版本，即 OpenJDK 11。**

如果可以得到社区的支持，我很高兴能够领导 JDK 8 更新项目和 JDK 11 更新项目。”

除了红帽以外，AWS 推出了 OpenJDK 长期支持版本 Amazon Corretto。阿里巴巴也开源了 OpenJDK 长期支持版本 Alibaba Dragonwell。

9.3 未来展望

Java 的变化速度从未如此之快——如今，该语言的新版本每六个月就会发布一次。

而之所以能够实现如此重大的转变，自然离不开一系列专注于**提高其性能与添加新功能**的协作性项目的贡献。这些项目的目标可谓雄心勃勃。正如 JetBrains 开发者布道师 Trisha Gee 在 QCon 伦敦 2019 大会上所言，“Java 即将迎来很多超酷的东西。”

而发展道路中的以下三大主要项目，将有助于确定 Java 的未来方向。

项目一：Loom 项目

尝试改进 Java 语言的并发处理方式，或者说是对于计算机在不同指令集执行之间切换能力的探索。

甲骨文公司 Loom 项目技术负责人 Ron Pressler 在 QCon 伦敦 2019 大会上向希望编写软件以处理并发任务的 Java 开发者们提出了两种都不够完美的选项：**要么编写无法通过扩展处理大量并发任务的“简单同步阻塞代码”，要么编写可扩展但编写难度极高且调试过程复杂的异步代码。**

为了寻求解决这个问题的方法，Loom 项目引入了一种将任务拆分为线程的新方法——所谓线程，即是指计算机在运行指令时的最小可能执行单元。在这方面，**Loom 引入了被称为 fibers 的新型轻量级用户线程。**

他在大会上指出，“**利用 fibers，如果我们确保其轻量化程度高于内核提供的线程，那么问题就得到了解决。大家将能够尽可能多地使用这些用户模式下的轻量级线程，且基本不会出现任何阻塞问题。**”

利用这些新的 fibers，用户将能够扩展 Java 虚拟机 (JVM) 以支持定界延续 (delimited continuations) 机制，从而使得指令集的执行实现暂停以及恢复。对这些延续进行暂停与恢复的任务将由 Java 中的 ForkJoinPool 调度程序以异步模式处理。

根据说明文档所言，**fibers 将使用与 Java 现有 Thread 类非常相似的 API**，这意味着 Java 开发人员的学习曲线应该不会太过陡峭。

项目二：Amber 项目

Amber 项目的目标，在于**支持“更小、面向生产力的 Java 语言功能”的开发，从而加快将新功能添加至 Java 语言中的速度。**

这套方案非常适合自 Java 9 以来，以更快速度持续发布的各 Java 新版本。

目前，以下 JDK 增强提案 (简称 JEP) 正在进行当中，并隶属于 Amber 项目之内。

生字符串：

生字符串使得开发人员能够更轻松地对文本进行适当格式化，且无需引入由转义字符带来的复杂性。

举例来说，开发人员不必使用转义字符来表示换行符，因此在以下字符串中：

```
1 | Hello world
```

可以直接编写为：

```
1 | `Hello world`
```

而非原本的：

```
1 | "Hello world "
```

该提案的说明文档中提到，这一变更将使得各类文本客串的输入变得更加简单，包括文件路径以及 SQL 语句等等。

如大家所见，生字符串应被包含在反引号之内。

用于 JDK API 的 Java 编译器 Intrinsic：

```
1 | https://openjdk.java.net/jeps/348
```

此项提案将允许开发人员对需要定期调用的重要代码段进行性能优化。

Pattern Matching：

```
1 | https://openjdk.java.net/jeps/305
```

Pattern Matching 能够简化利用 Java 中 instanceof 运算符检查对象是否属于特定类的过程，而后提取该对象的组件以进行进一步处理。

如此一来，以下操作语法：

```
1 | if (obj instanceof Integer) { int intValue = ((Integer) obj).intValue(); // use  
  |   intValue }
```

将可被简化为：

```
1 | if (x instanceof Integer i) { // can use i here, of type Integer }
```

Switch Expressions：

```
1 | https://openjdk.java.net/jeps/325
```

Switch expressions 已经在 Java 12 当中以预览版形式推出，允许开发人员利用更简单的语法通过 switch 语句为输入内容指定不同的响应方式。

举例来说，现在我们不再需要始终在以下语法当中使用 switch 语句：

```
1 | switch (port) { case 20: type = PortType.FTP; break; }
```

而可以采取以下更为简洁的表达方式：

```
1 | Switch (port) { case 20 -> PortType.FTP; }
```

项目三：Valhalla 项目

Valhalla 项目专注于支持“高级”JVM 与语言功能的开发。

目前 Valhalla 项目的候选提案还比较有限，具体包括：

Value Types：

```
1 | https://openjdk.java.net/jeps/169
```

此项提案旨在允许 JVM 处理一种新的类型，即 Value Types。

这些新的不可变类型将拥有与 int 等基元类似的内存效率，但同时又与普通类一样能够保存一整套基元集合。提案说明文档中指出，其目标在于“为 JVM 基础设施提供处理不可变与无引用对象的能力，从而实现使用非基元类型进行高效按值计算的目标。”

Generic Specialization：

```
1 | https://openjdk.java.net/jeps/218
```

此项提案扩展了适用于泛型的具体类型，其中包括基元以及即将推出的 Value Types。



尚硅谷官网：<http://www.atguigu.com/>